Guia De Desenvolvimento
Rest
Spring MVC
Spring Boot
Git
2026-2
Parte 4 –
Criando a API com Spring Web

Introdução

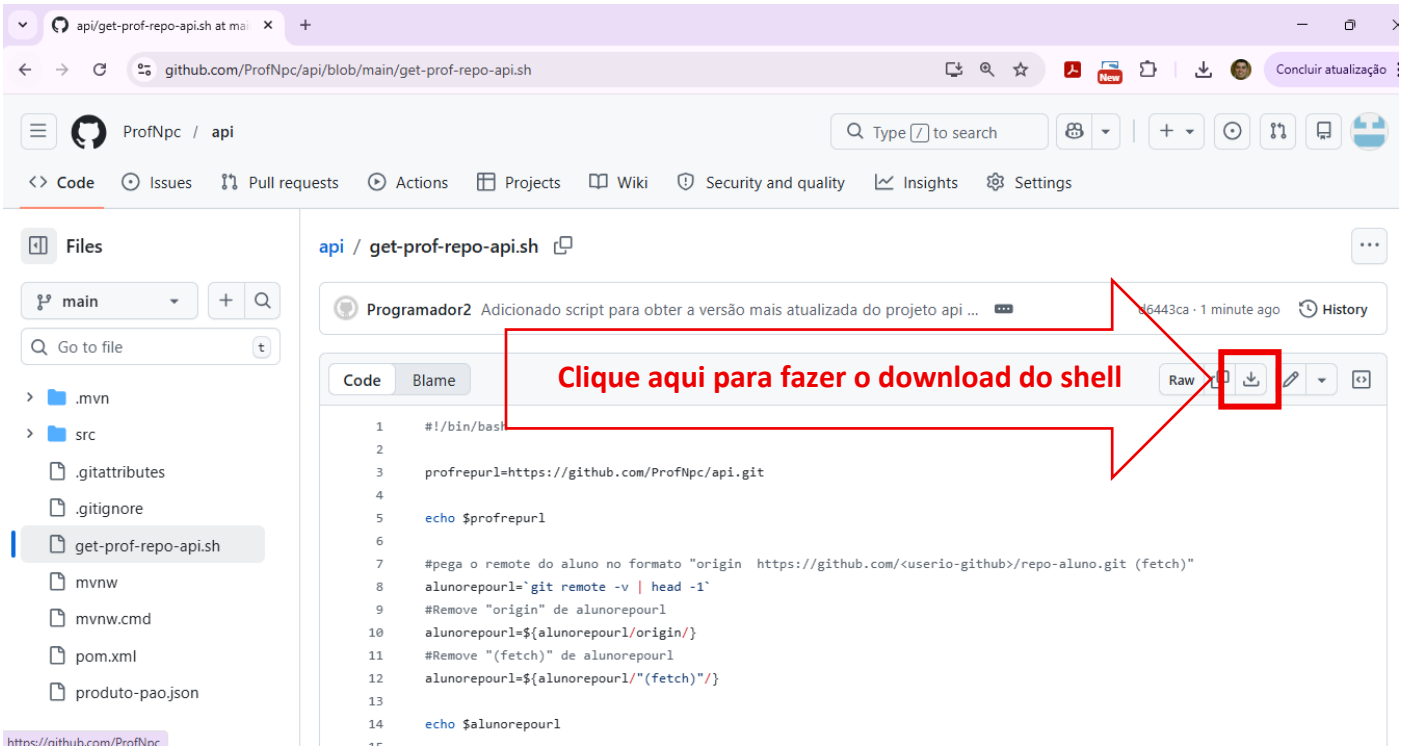
Neste documento teremos o passo-a-passo para a construção de uma aplicação web/rest utilizando as ferramentas Spring MVC, Spring Boot e Git.

Este guia se destina a dar suporte as aulas de LIP1 do segundo módulo do curso de informática noturno do ITB Belval, portanto, algumas explicações mais detalhadas serão omitidas pois serão apresentadas pelo professor durante a aula.

O projeto desenvolvido aqui funciona como referência para a criação do *BackEnd* dos projetos de TCC. A cada aula nós avançaremos com o desenvolvimento desse projeto, salvando seu progresso no repositório público https://github.com/ProfNpc/api.

Caso você venha a perder alguma aula, poderá obter a versão mais atualizada feita pelo professor e seguir acompanhando as aulas de onde a turma parou. Para isto basta colocar o *shell script* https://github.com/ProfNpc/api/blob/main/get-prof-repo-api.sh na raiz do projeto api e executá-lo em seguida.

Nesta parte criaremos a camada de **Controller** da nossa API REST, expondo os endpoints HTTP para as operações de criação, leitura, atualização e exclusão (**CRUD**) de produtos, utilizando as anotações do **Spring Web**.



Parte 5

O que é um Controller em uma API REST?

Em uma API REST construída com Spring, o **Controller** é a classe responsável por receber as requisições HTTP enviadas pelos clientes (navegadores, aplicativos, outros servidores) e devolver as respostas apropriadas. Ele é o ponto de entrada da aplicação para o mundo externo.

A anotação `@RestController` combina duas anotações em uma só: `@Controller`, que registra a classe como um componente Spring capaz de tratar requisições web, e `@ResponseBody`, que instrui o Spring a serializar automaticamente os objetos Java retornados pelos métodos em JSON para o corpo da resposta HTTP.

A anotação `@RequestMapping` aplicada à classe define o prefixo de URL comum a todos os endpoints daquele **Controller**. Por exemplo, `@RequestMapping("/produtos")` faz com que todos os métodos da classe respondam a URLs que começam com `/produtos`.

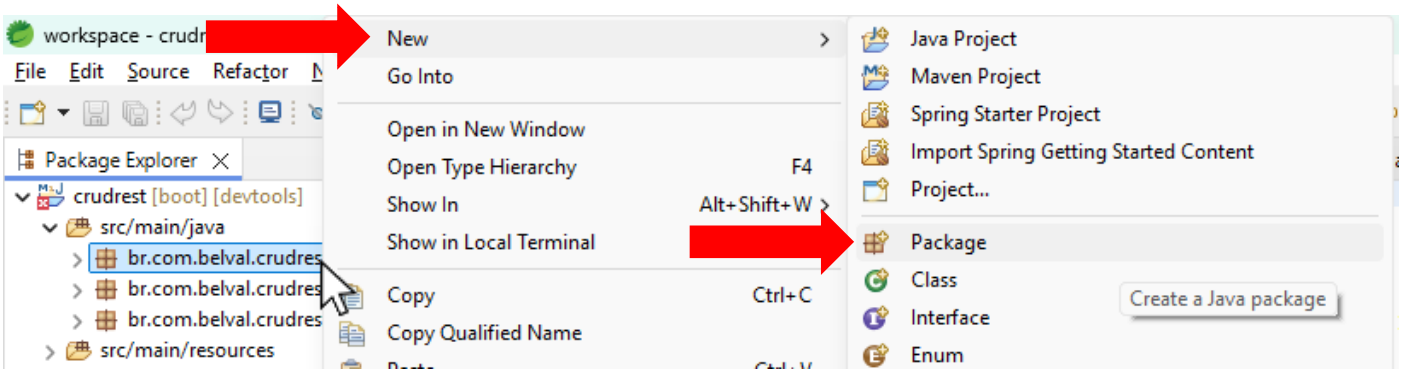
Verbos HTTP e anotações correspondentes

Cada operação CRUD é mapeada a um verbo HTTP diferente. O Spring fornece anotações específicas para cada um deles:

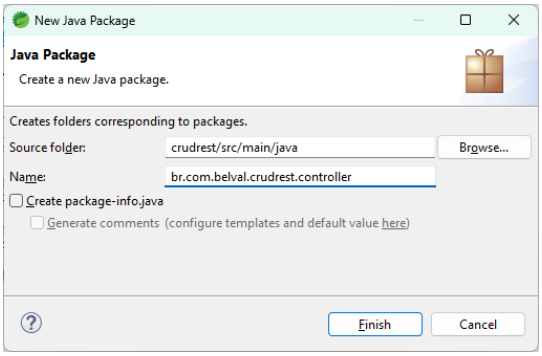
Verbo HTTP	Anotação Spring	Uso típico
POST	@PostMapping	Criar um novo recurso. O corpo da requisição contém os dados do novo objeto.
GET	@GetMapping	Buscar um ou mais recursos. Não altera dados no servidor.
PUT	@PutMapping	Substituir completamente um recurso existente pelo objeto enviado no corpo.
DELETE	@DeleteMapping	Remover um recurso identificado pelo ID informado na URL.
PATCH	@PatchMapping	Alterar parte dos atributos de um recurso existente pelos valores dos atributos enviados no corpo da requisição.

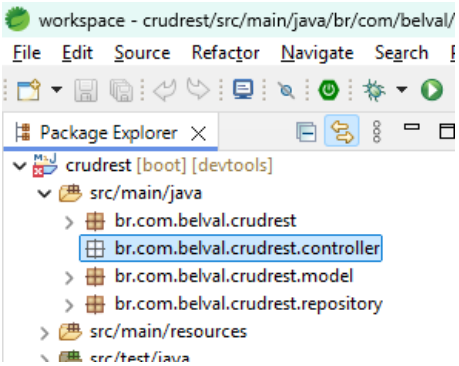
1 — Criando a classe ProdutoController

1.1 – Clique com o botão direito do mouse sobre `br.com.belval.crudrest` >> **New** >> **Package**

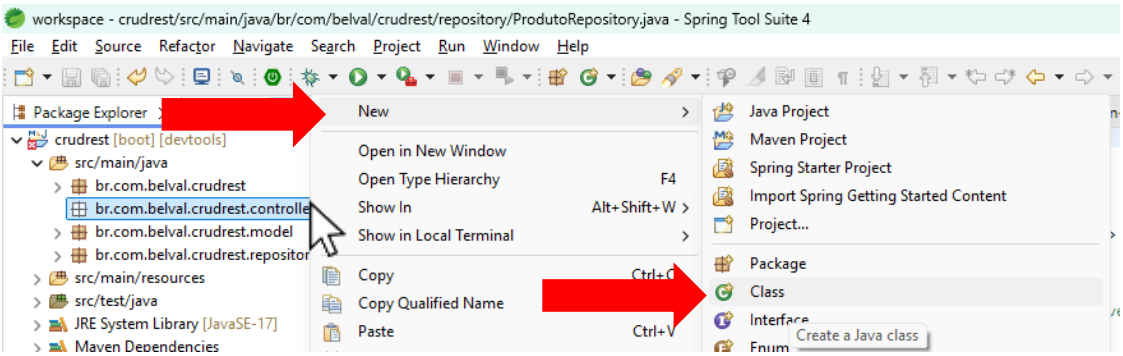


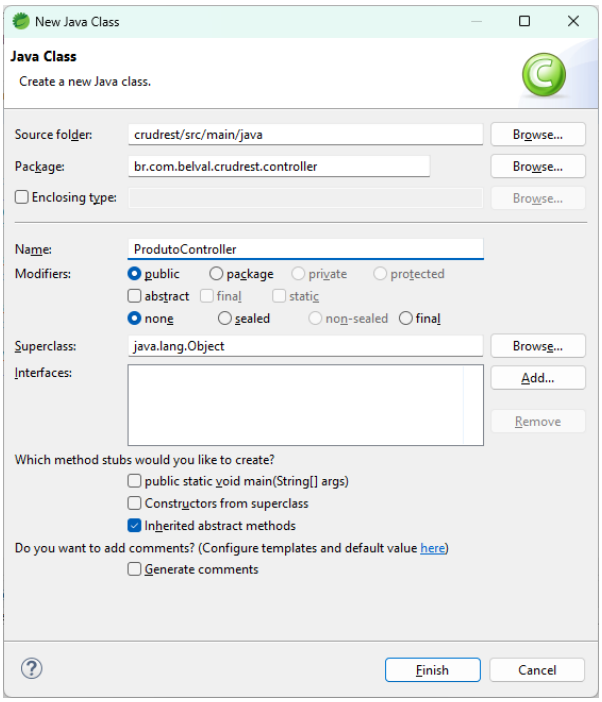
1.2 - Na janela que se abre, no campo “Name”, adicione ao final do pacote da classe principal “`.controller`”, deixando o campo com o valor de **`br.com.belval.crudrest.controller`** e clique em “Finish”.

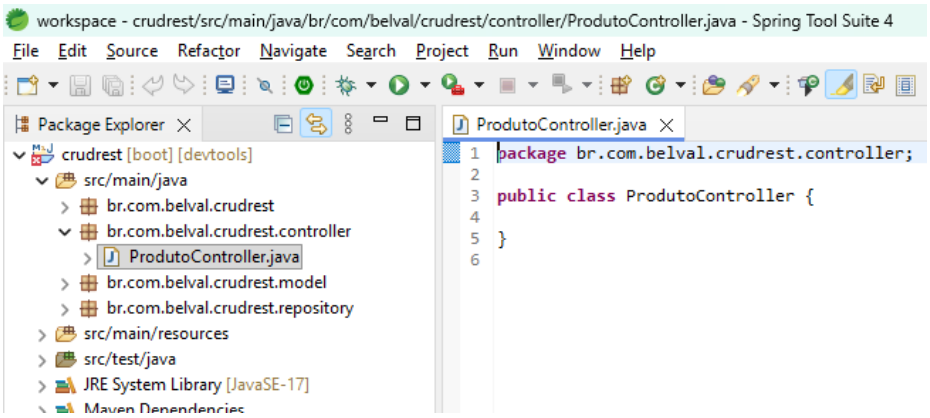




1.3 – Em seguida crie a classe: clique com o botão direito sobre o pacote br.com.belval.crudrest.controller >> **New** >> **Class**, preencha o campo "Name" com ProdutoController e clique em **Finish**.







2 — Anotando o Controller e injetando o ProdutoRepository

2.1 – Edite a classe ProdutoController para adicionar as anotações @RestController e @RequestMapping, e declare o atributo produtoRepository que será injetado automaticamente pelo Spring via @Autowired:

```
package br.com.belval.crudrest.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import br.com.belval.crudrest.repository.ProdutoRepository;

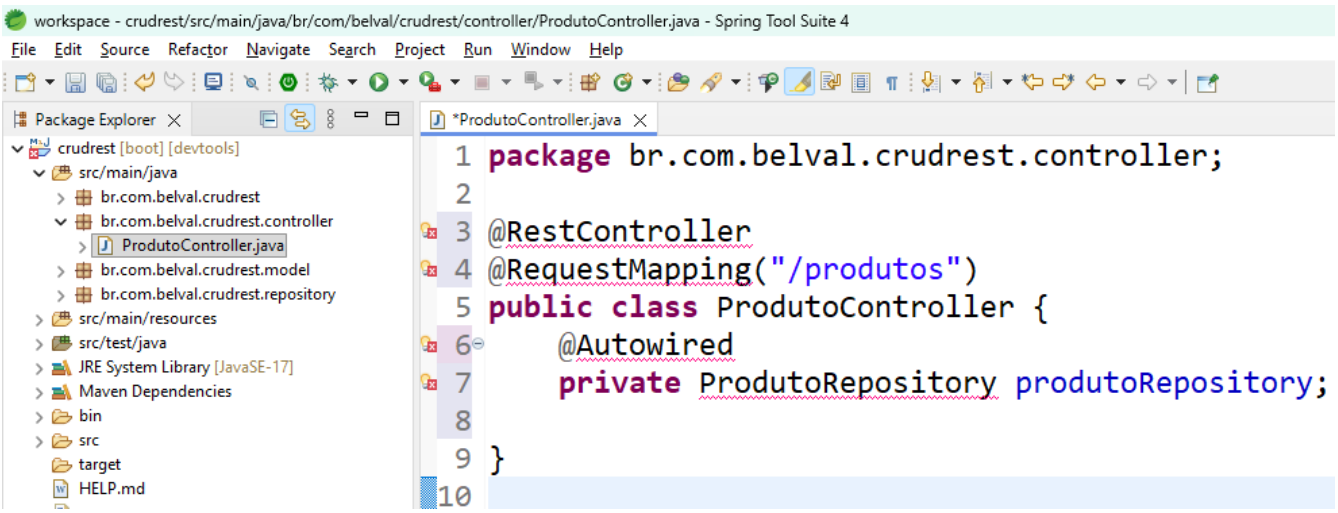
@RestController
@RequestMapping("/produtos")
public class ProdutoController {

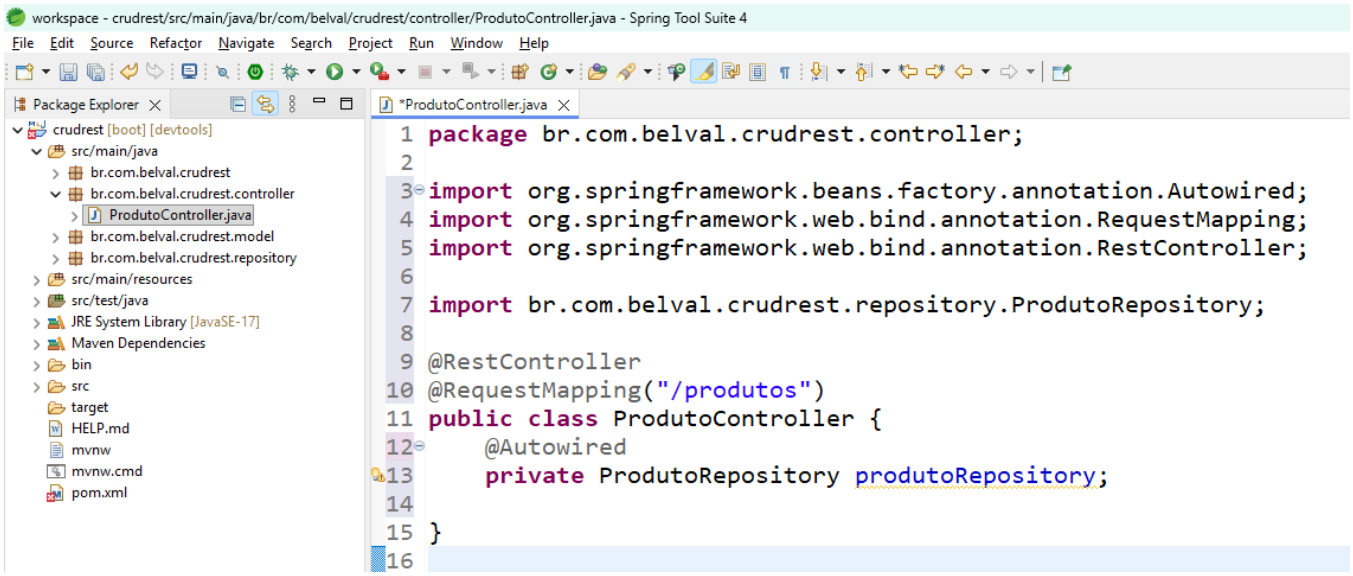
    @Autowired
    private ProdutoRepository produtoRepository;

}
```

A anotação @Autowired instrui o container Spring a criar e injetar um objeto que atenda à interface ProdutoRepository no atributo produtoRepository quando ProdutoRepository for criado. Esse objeto será criado pelo Spring e qualquer coisa que ele precise para ser instanciado, como uma conexão com o banco de dados, será fornecido a ele pelo Spring. Não é necessário usar new — o Spring cuida disso.

2.2 – Salve as alterações e caso algum erro aparece, veja abaixo, não esqueça de adicionar os import's com CTRL + SHIFT + O e salve o arquivo novamente.





3 — Método criarProduto (POST)

O método criarProduto receberá os dados do novo produto no corpo da requisição HTTP (formato JSON), salvará o produto no banco por meio do produtoRepository e devolverá o produto recém-criado com o ID gerado pelo banco. O código de resposta HTTP 201 Created indica ao cliente que o recurso foi criado com sucesso.

```
...
@RestController
@RequestMapping("/produtos")
public class ProdutoController {
    ....
    //curl -X POST http://localhost:8080/produtos -H "Content-Type: application/json; Charset=utf-8" -d @produto-pao.json
    @PostMapping
    public ResponseEntity<Produto> criarProduto(@RequestBody Produto produto) {

                produto.setId(null);
                produto.setDataCriacao(LocalDateTime.now());

                System.out.println("Chegou : " + produto.toString());

                Produto salvo = produtoRepository.save(produto);

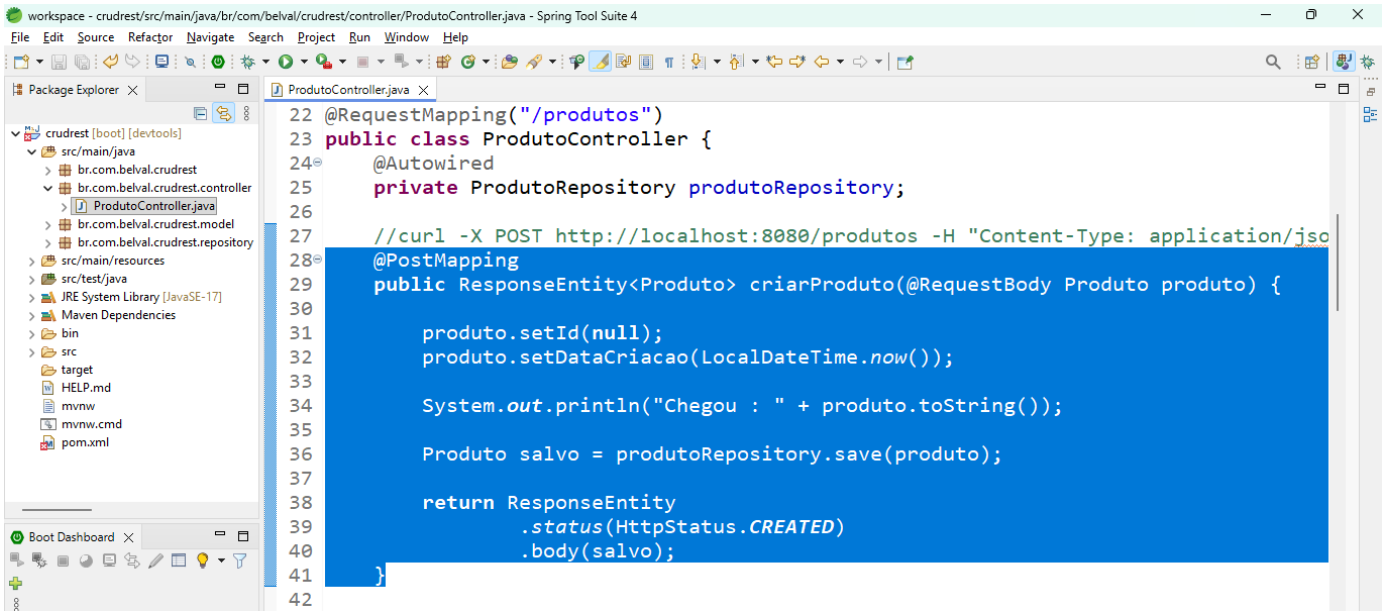
                return ResponseEntity
                                .status(HttpStatus.CREATED)
                                .body(salvo);
    }

}
```

A anotação @RequestBody instrui o Spring a desserializar automaticamente o JSON recebido no corpo da requisição e transformá-lo em um objeto Produto. O método produtoRepository.save(produto) persiste o objeto no banco e retorna a instância com o id preenchido pelo banco de dados.

A instrução produto.setId(null); define o valor do atributo id para null para garantir que o método save(), quando chamado, crie um novo produto, ou seja, execute um comando INSERT na tabela produto. Se isso não fosse feito, o cliente poderia chamar o método criarProduto() informando um valor de id de um produto já presente na tabela e isso faria com que a chamada da save() provocasse a execução de um UPDATE.

A instrução produto.setDataCriacao(LocalDateTime.now()); define a data de criação com a data corrente do computador que roda a aplicação.

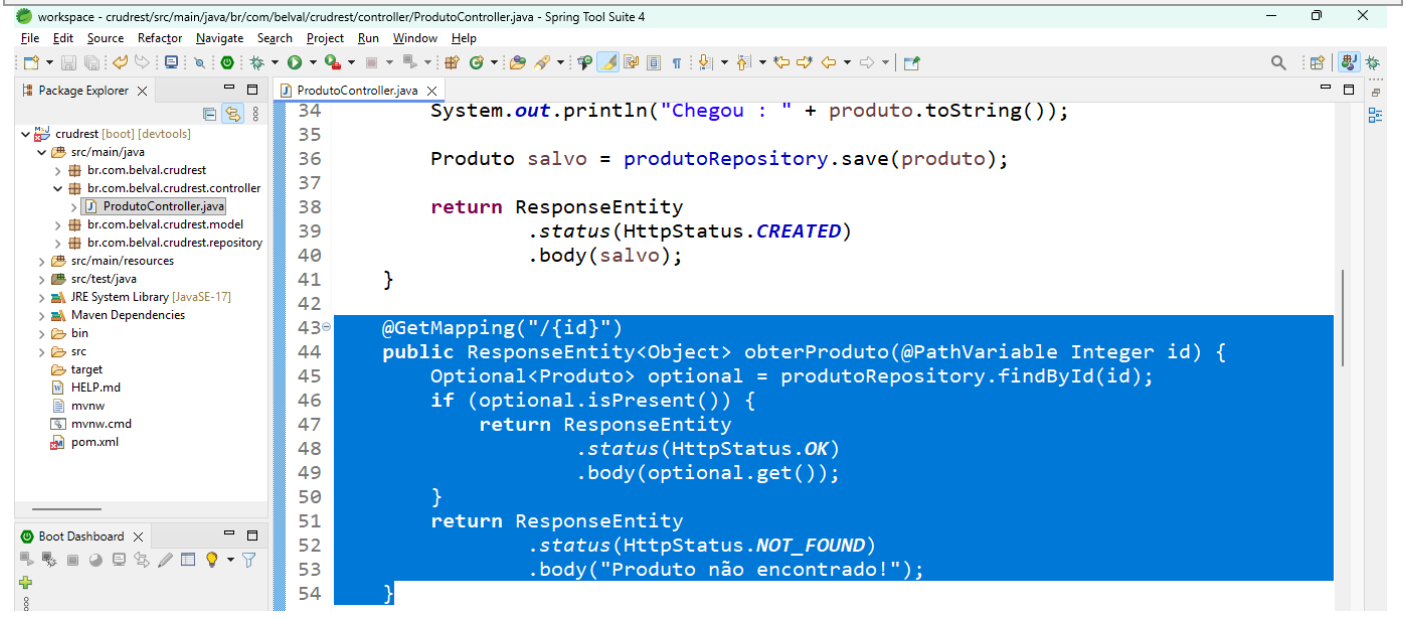


4 — Método obterProduto (GET por ID)

O método obterProduto busca um único produto no banco pelo seu id. O id é recebido como parte da URL (*path variable*). Se o produto não for encontrado, retornamos HTTP 404 Not Found:

```
...
@RestController
@RequestMapping("/produtos")
public class ProdutoController {
    ....
    @GetMapping("/{id}")
    public ResponseEntity<Object> obterProduto(@PathVariable Integer id) {
        Optional<Produto> optional = produtoRepository.findById(id);
        if (optional.isPresent()) {
            return ResponseEntity
                        .status(HttpStatus.OK)
                        .body(optional.get());
        }
        return ResponseEntity
                .status(HttpStatus.NOT_FOUND)
                .body("Produto não encontrado!");
    }

}
```



A anotação @GetMapping("/{id}") mapeia requisições do tipo GET /produtos/1 para este método. O {id} na URL é um *path variable*: um trecho dinâmico da URL capturado pela anotação @PathVariable e passado como parâmetro ao método.

A chamada de método produtoRepository.findById(id); executa um SELECT * FROM produto WHERE id = <id> e retorna um objeto do tipo Optional<Produto>. Um Optional<Produto> pode ou não ter um objeto Produto a depender se existe ou não um produto com o id recebido como parâmetro na tabela produto.

O método obterProduto retorna um ResponseEntity<Object>. Um ResponseEntity é um objeto que representa uma resposta HTTP que inclui um código de retorno e um corpo que pode ser um objeto qualquer, no nosso caso poderá ser um Produto ou uma String com a mensagem "Produto não encontrado!" caso nenhum Produto seja encontrado.

Chamamos optional.isPresent() para verificar se dentro de optional existe um Produto ou não, o que será verdade apenas se o produto com o id pesquisado existir no banco. Se existir, optional.get() retornará o objeto e usaremos ResponseEntity.status(HttpStatus.**OK**).body(optional.get()); para devolver a requisição HTTP com o código HTTP 200 (Sucesso). Caso não exista um produto dentro de optional, ResponseEntity .status(HttpStatus.***NOT_FOUND***).body("Produto não encontrado!"); devolve HTTP 404 e no corpo a mensagem "Produto não encontrado!".

5 — Método listarProdutos (GET todos)

O método listarProdutos retorna todos os produtos cadastrados no banco. Por não receber um id na URL, ele é mapeado para GET /produtos (o mesmo prefixo da classe, sem caminho adicional):

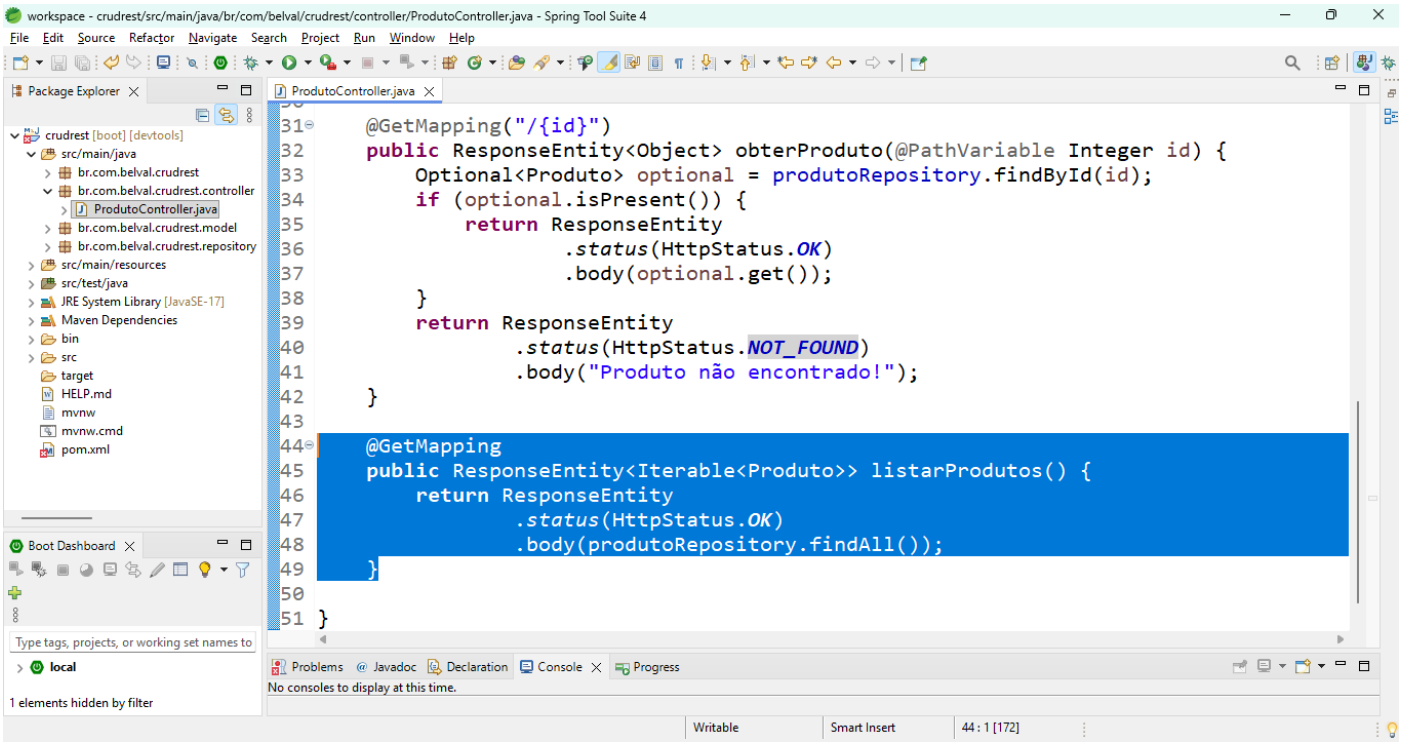
```
...
@RestController
@RequestMapping("/produtos")
public class ProdutoController {

	...
	@GetMapping
	public ResponseEntity<Iterable<Produto>> listarProdutos() {
		return ResponseEntity
			.status(HttpStatus.OK)
			.body(produtoRepository.findAll());
	}
}
```

O método findAll() do CrudRepository retorna um Iterable<Produto>. Um Iterable é um objeto que representa uma coleção que podemos percorrer, assim como um List.

"Iterar", segundo o dicionário, significa **"repetir um processo, uma ação ou um conjunto de instruções com o objetivo de se aproximar de um resultado desejado ou de aprimorar algo aos poucos".** Para percorrer os elementos de uma lista precisamos repetir o processo de "ir para o próximo".

"Iterable" seria algo como "iterável" ou, no caso de uma lista, "percorrível".



6 — Método atualizarProduto (PUT)

O método atualizarProduto recebe o id do produto a ser atualizado na URL e os novos dados no corpo da requisição. Primeiro verificamos se o produto existe; se não existir, retornamos HTTP 404. Se existir, copiamos os novos valores para o objeto recuperado do banco e salvamos:

```
...
@RestController
@RequestMapping("/produtos")
public class ProdutoController {

	...
	//curl -X PUT http://localhost:8080/produtos/1 -H "Content-Type: application/json; Charset=utf-8" -d @produto-pao2.json
	@PutMapping("/{id}")
	public ResponseEntity<Object> atualizarProduto(
			@PathVariable Integer id,
			@RequestBody Produto produtoNovo) {

		Optional<Produto> optional = produtoRepository.findById(id);

		if (optional.isEmpty()) {
			return ResponseEntity
				.status(HttpStatus.NOT_FOUND)
				.body("Produto não encontrado!");
		}

		Produto produtoAtual = optional.get();

		//São sobreescritos todos os atributos menos
		//id e dataCriacao, estes são mantidos.
		//Atualiza apenas os campos mutáveis.
		produtoAtual.setNome(produtoNovo.getNome());
		produtoAtual.setDescricao(produtoNovo.getDescricao());
		produtoAtual.setPreco(produtoNovo.getPreco());

		produtoRepository.save(produtoAtual);

		return ResponseEntity
			.status(HttpStatus.OK)
			.body("Produto atualizado com sucesso!");
	}
}
```

*ProdutoController.java

```
                .body(produtoRepository.findAll());
    }

    //curl -X PUT http://localhost:8080/produtos/1 -H "Content-Type: application/json; Charset=utf-8" -d @produto-pao2.json
    @PutMapping("/{id}")
    public ResponseEntity<Object> atualizarProduto(
            @PathVariable Integer id,
            @RequestBody Produto produtoNovo) {

        Optional<Produto> optional = produtoRepository.findById(id);

        if (optional.isEmpty()) {

            return ResponseEntity
                    .status(HttpStatus.NOT_FOUND)
                    .body("Produto não encontrado!");
        }

        Produto produtoAtual = optional.get();

        //São sobreescritos todos os atributos menos
        //id e dataCriacao, estes são mantidos.
        //Atualiza apenas os campos mutáveis.
        produtoAtual.setNome(produtoNovo.getNome());
        produtoAtual.setDescricao(produtoNovo.getDescricao());
        produtoAtual.setPreco(produtoNovo.getPreco());

        produtoRepository.save(produtoAtual);

        return ResponseEntity
                .status(HttpStatus.OK)
                .body("Produto atualizado com sucesso!");
    }
```

É importante recuperar o produto do banco com findById e atualizar os campos um a um no objeto gerenciado pelo JPA, em vez de simplesmente salvar o objeto recebido no corpo da requisição. Isso garante que apenas os campos mutáveis sejam alterados, já os campos imutáveis como dataCriacao e id tenham seus valores iniciais preservados.

7 — Método apagarProduto (DELETE)

O método apagarProduto recebe o id na URL, verifica se o produto existe e o remove do banco. Em caso de sucesso retornamos HTTP 204 No Content (HttpStatus.NO_CONTENT): sem corpo de resposta. Se o produto não existir, HttpStatus.NOT_FOUND com HTTP 404:

```
...
@RestController
@RequestMapping("/produtos")
public class ProdutoController {

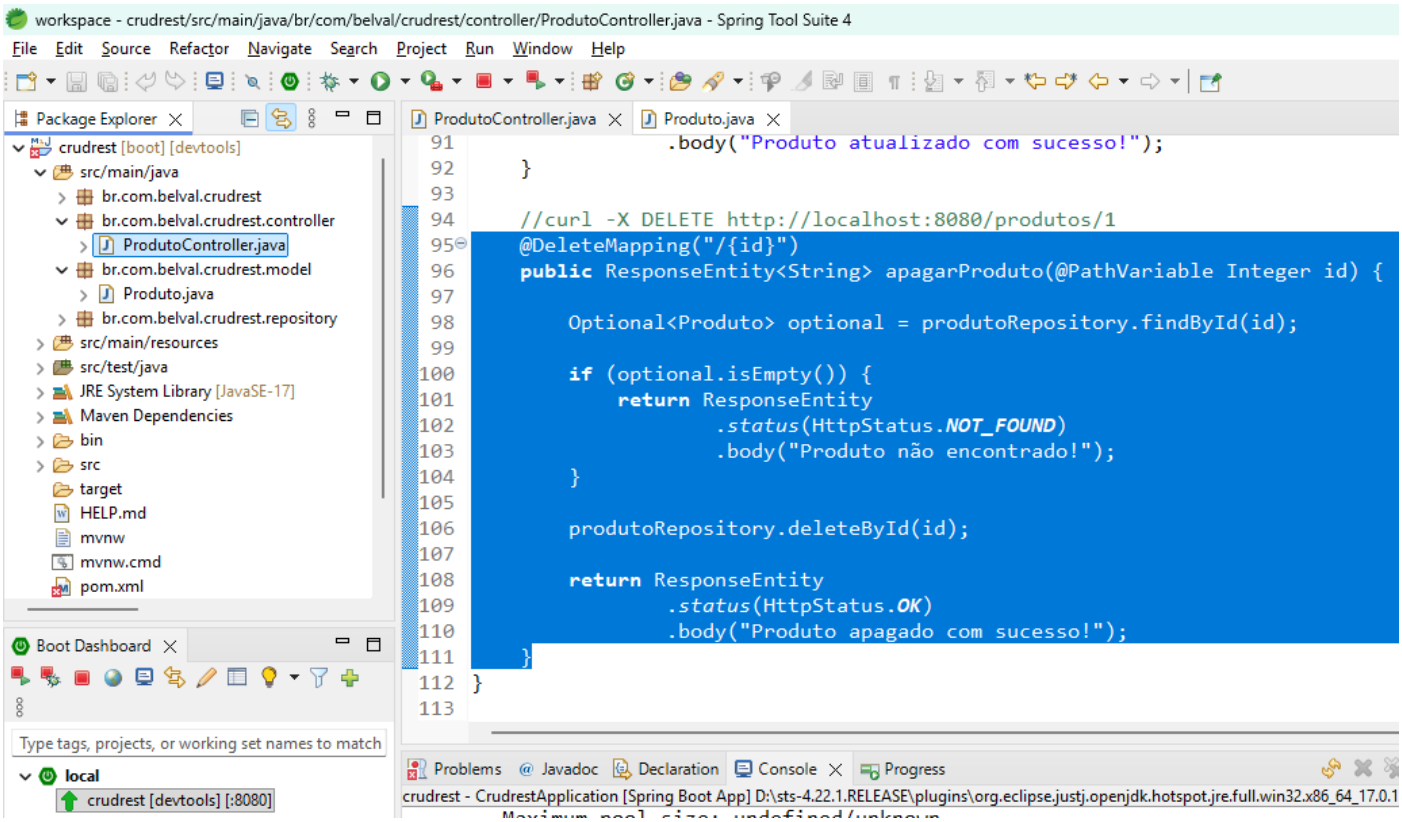
        ...
        //curl -X DELETE http://localhost:8080/produtos/1
        @DeleteMapping("/{id}")
        public ResponseEntity<String> apagarProduto(@PathVariable Integer id) {

                Optional<Produto> optional = produtoRepository.findById(id);

                if (optional.isEmpty()) {
                        return ResponseEntity
                                .status(HttpStatus.NOT_FOUND)
                                .body("Produto não encontrado!");
                }

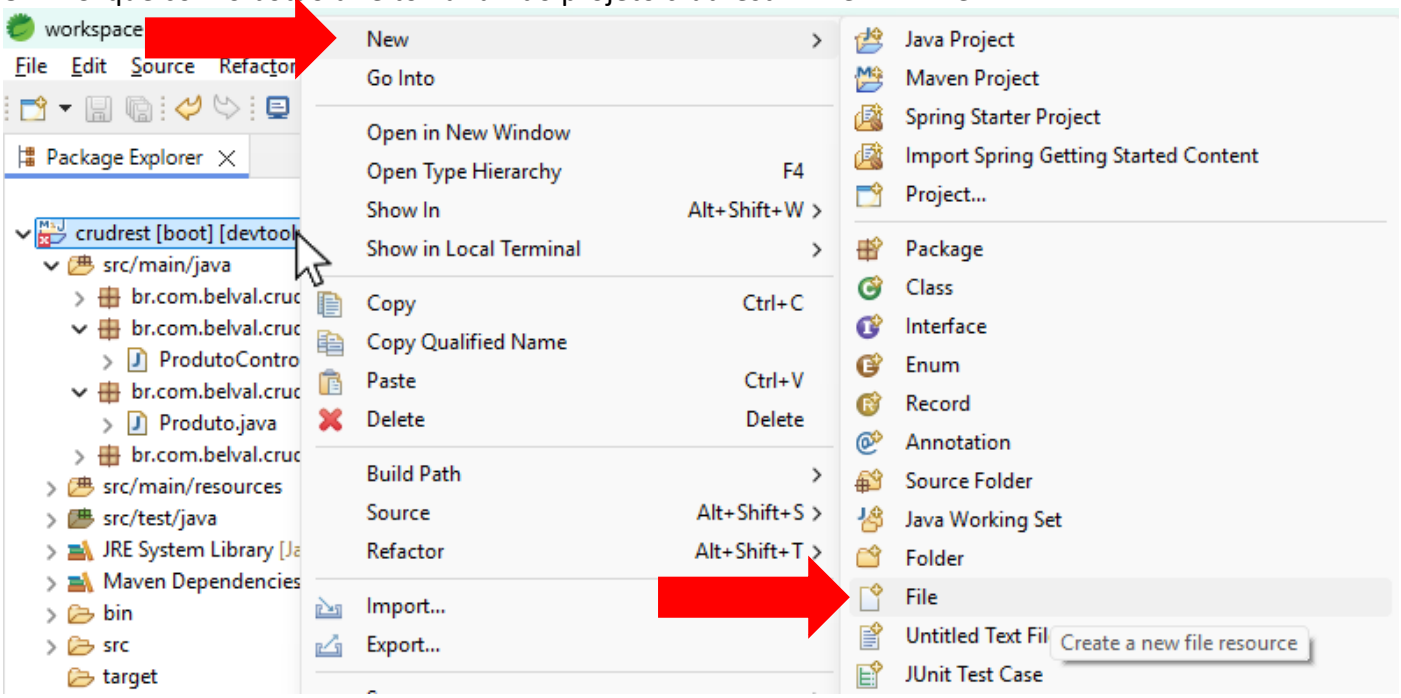
                produtoRepository.deleteById(id);

                return ResponseEntity
                        .status(HttpStatus.OK)
                        .body("Produto apagado com sucesso!");
        }
}
```

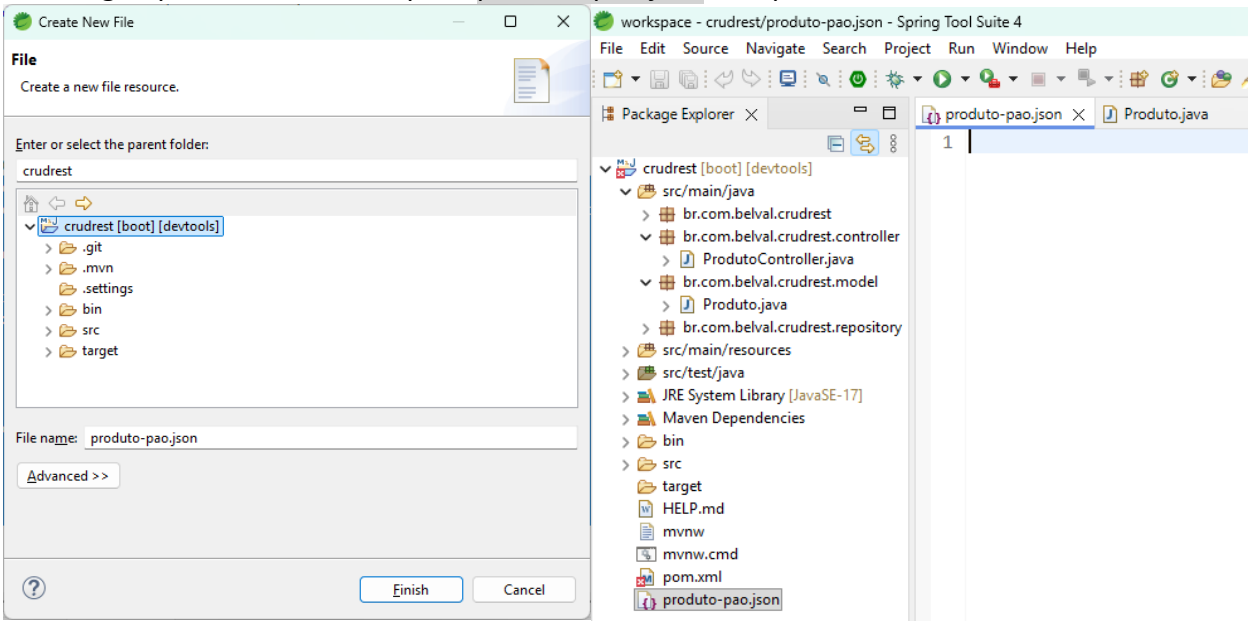


8 — Testando ProdutoController

8.1 – Clique com o botão direito na raiz do projeto crudrest >> New >> File



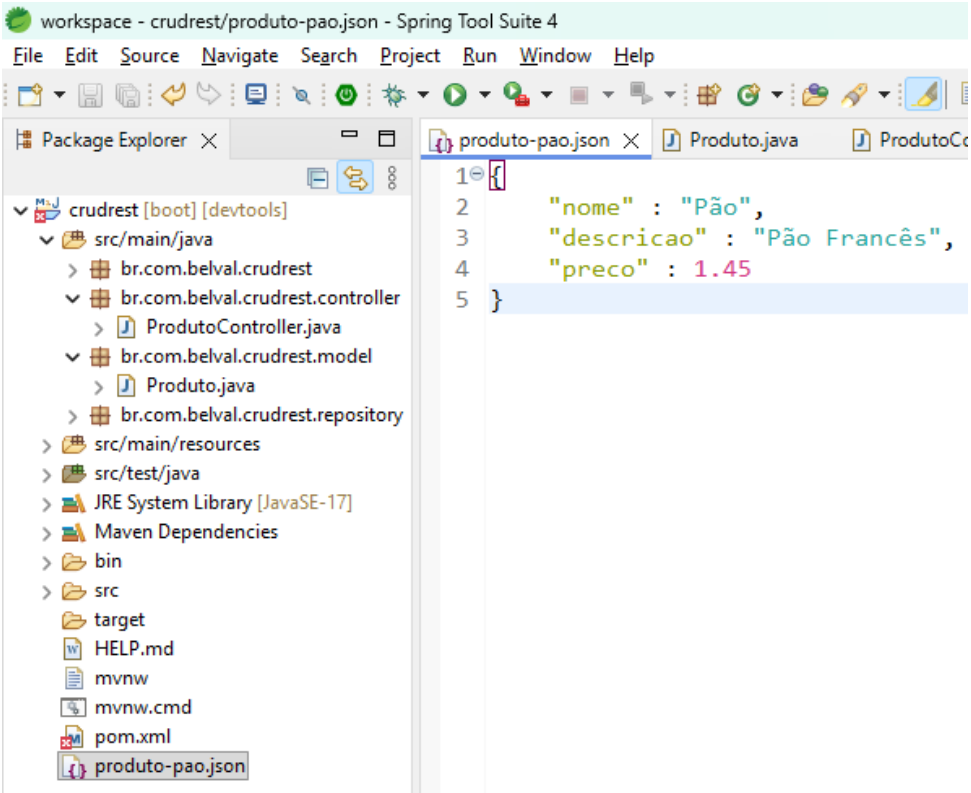
8.2 – Digite para o nome do arquivo produto-pao.json e clique em “Finish”



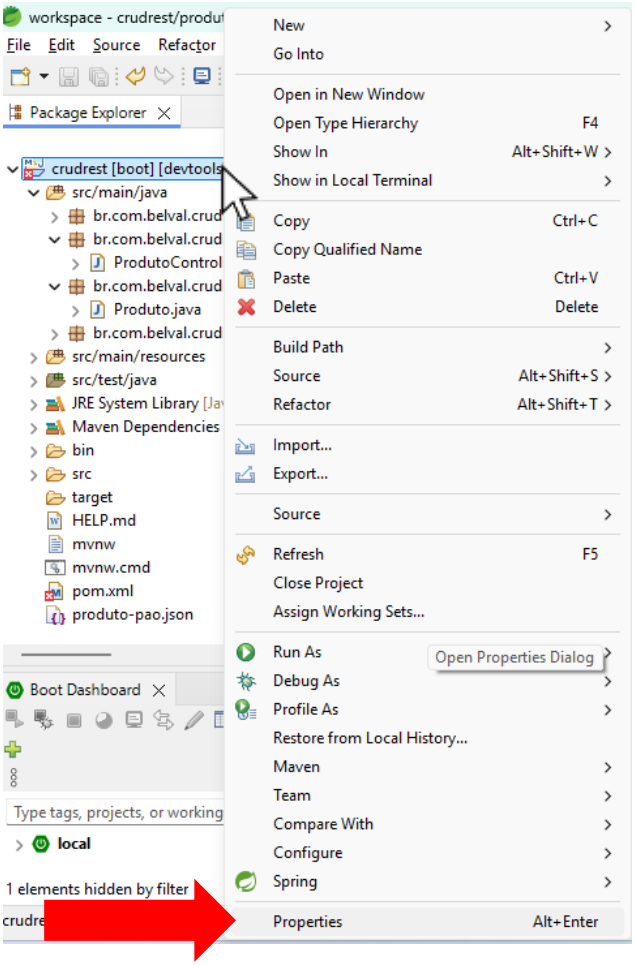
8.3 – Dentro do arquivo vamos inserir a descrição de um objeto produto utilizando uma notação, uma forma de escrever, que vem do JavaScript, daí vem o nome “json”, **J**ava**S**cript **O**bject **N**otation, que seria algo como notação de objeto em JavaScript.

Arquivo **produto-pao.json**

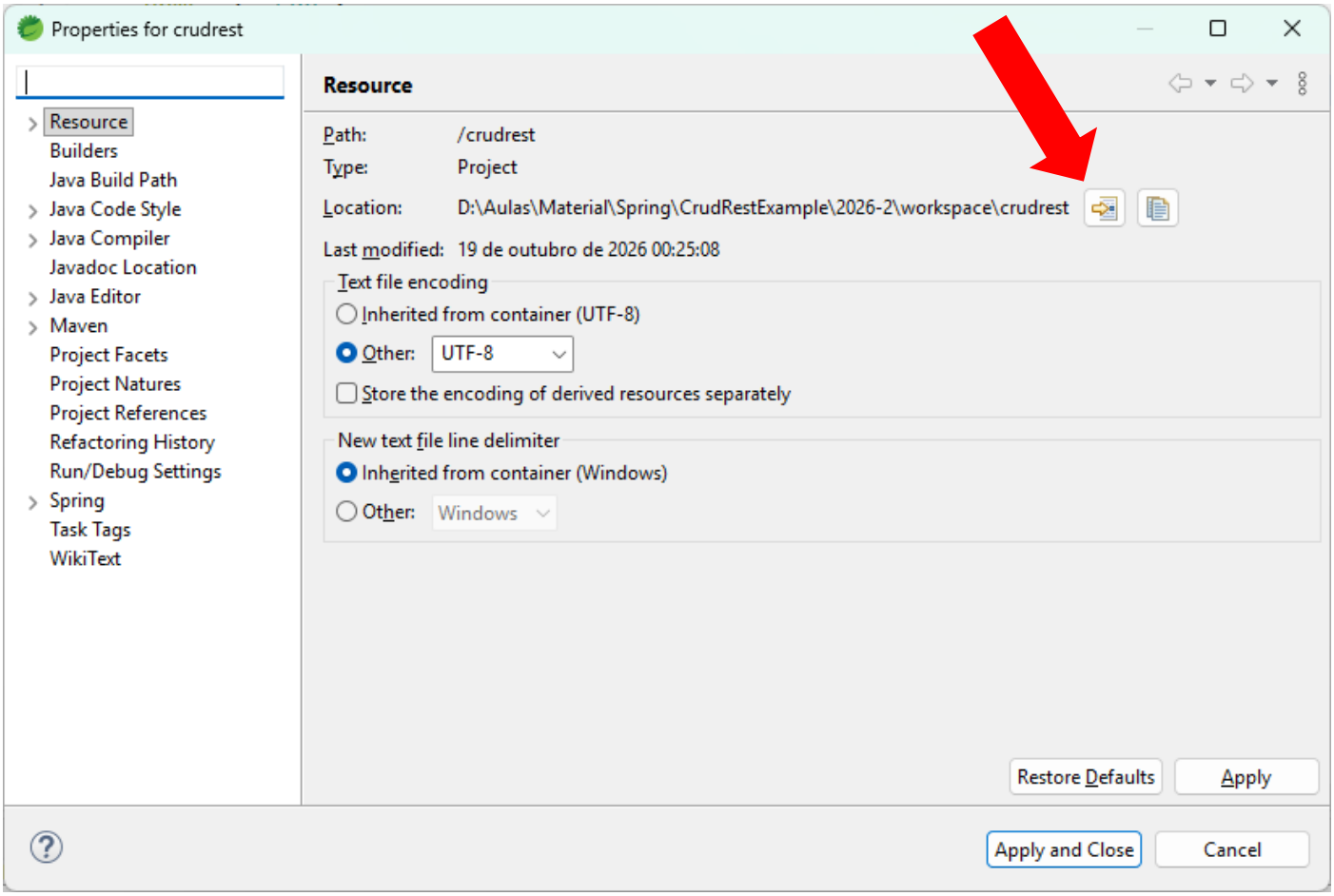
```
{
	"nome" : "Pão",
	"descricao" : "Pão Francês",
	"preco" : 1.45
}
```

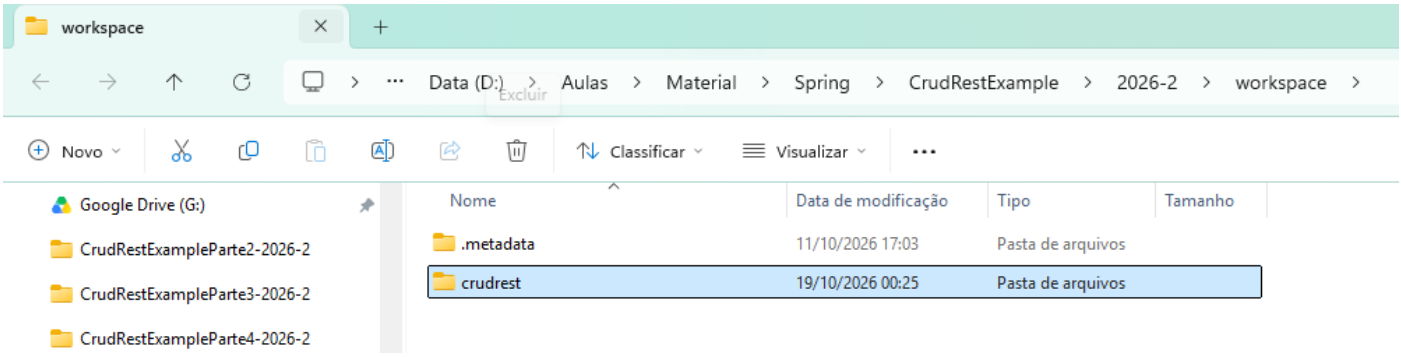


8.4 – Clique com o botão direito do mouse na raiz do projeto >> Properties

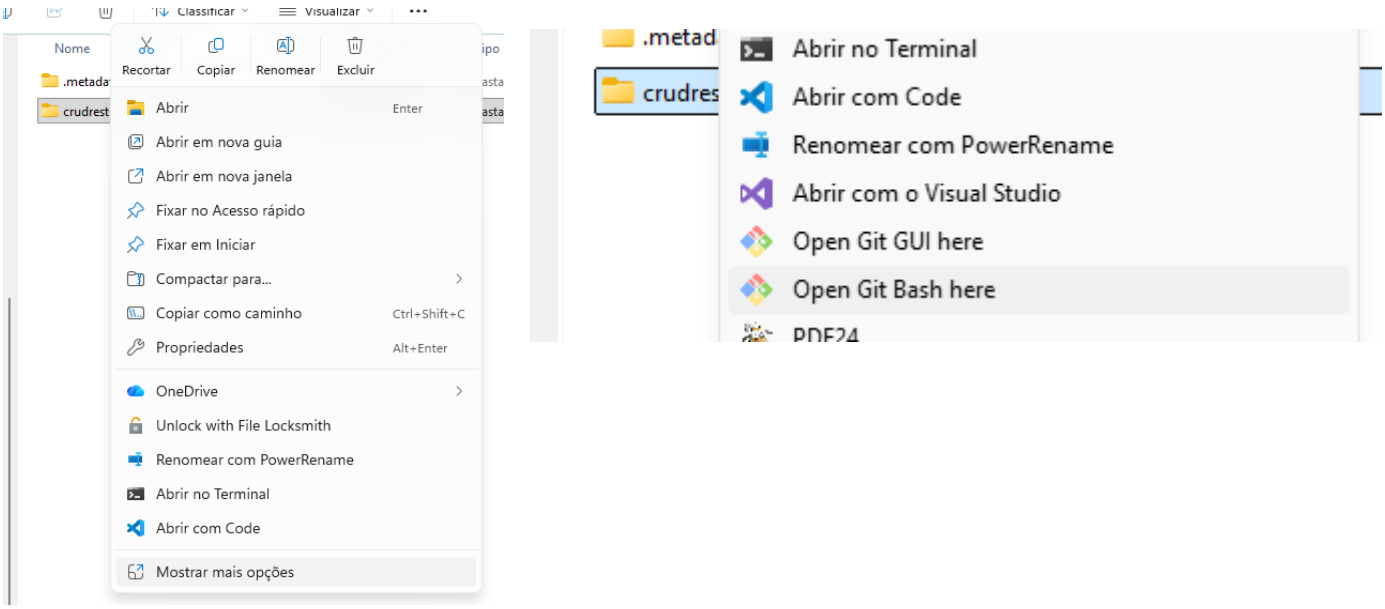


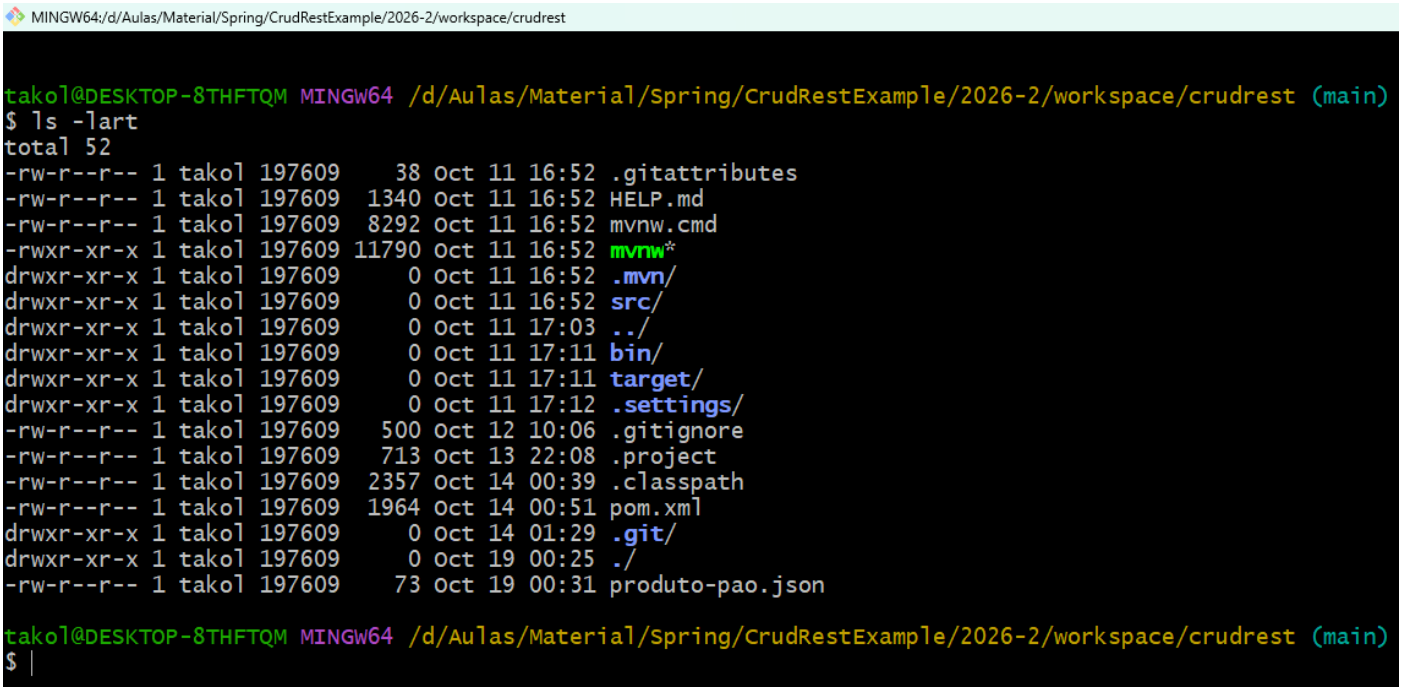
8.5 – Clique no botão para abrir o diretório do projeto 





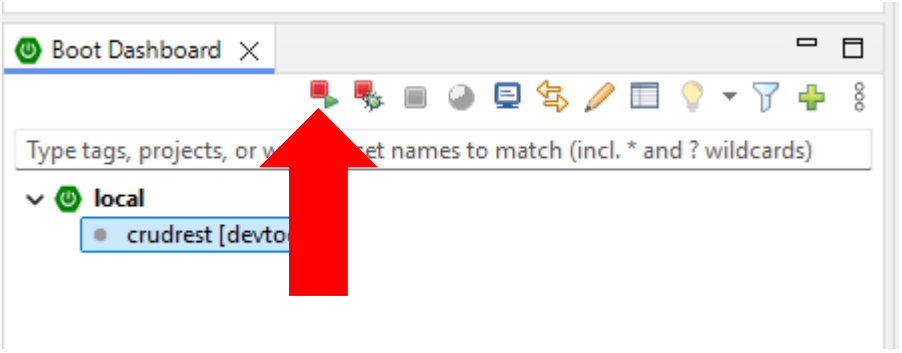
8.6 – Clique com o botão direito do mouse sobre o diretório “crudrest”, clique em “Mostrar mais opções” >> “Open Git Bash here”

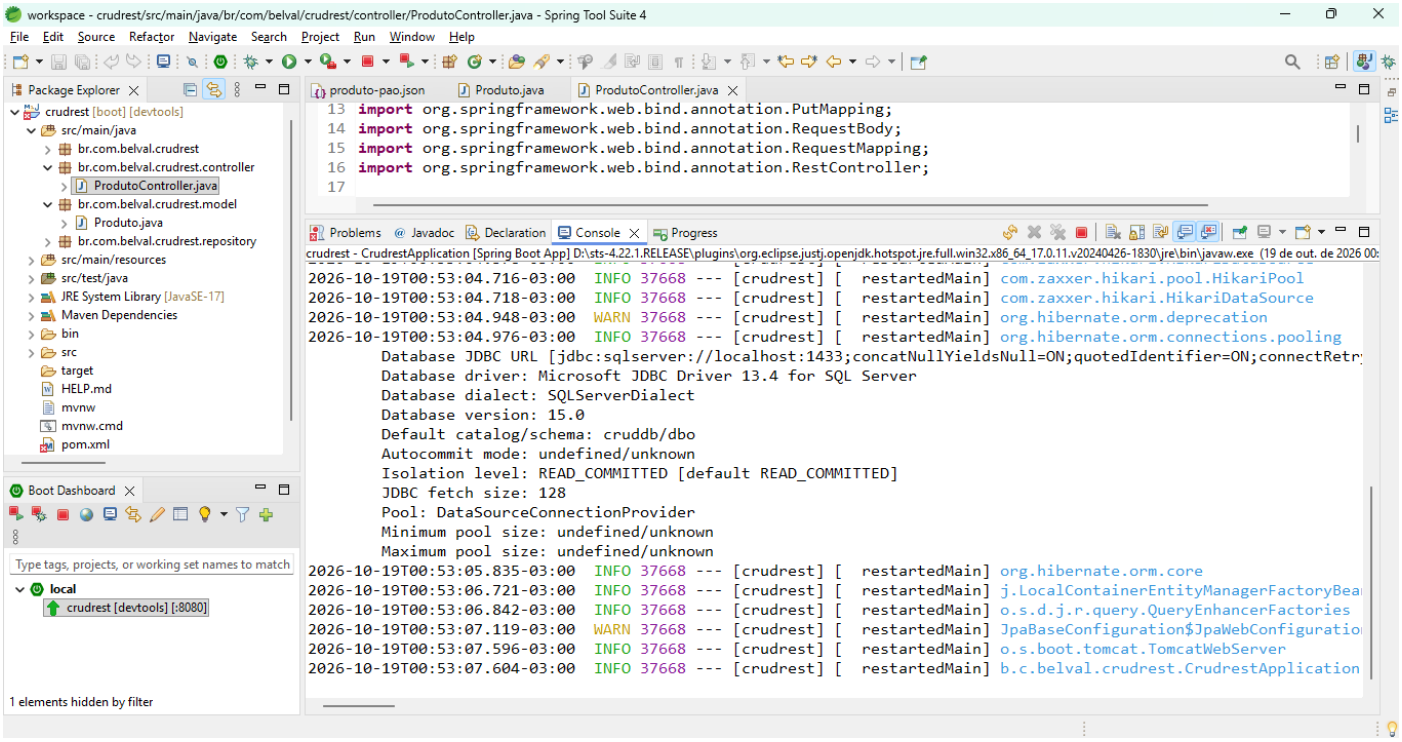




Observe que “crudrest” é o diretório onde está o arquivo produto-pao.json.

8.7 – Na view “Boot Dashboard”, selecionar o projeto e clicar no botão “Start or Restart”.



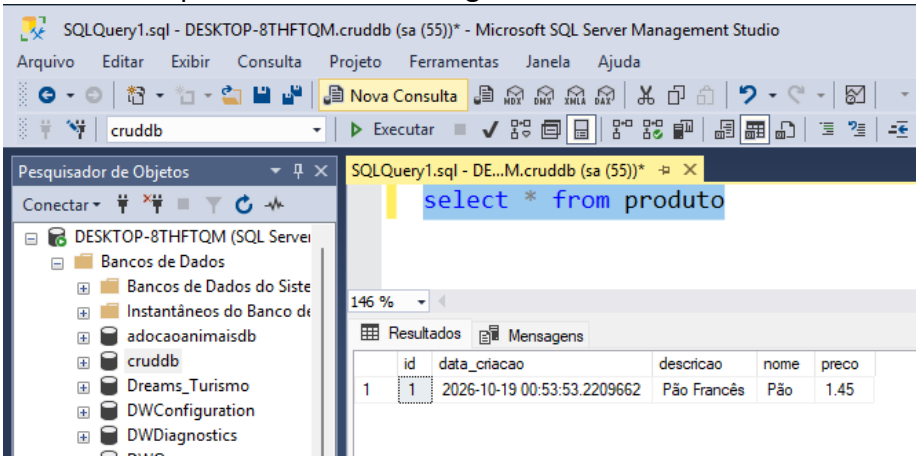


8.8 – Digite o seguinte comando no Git Bash e dê enter:

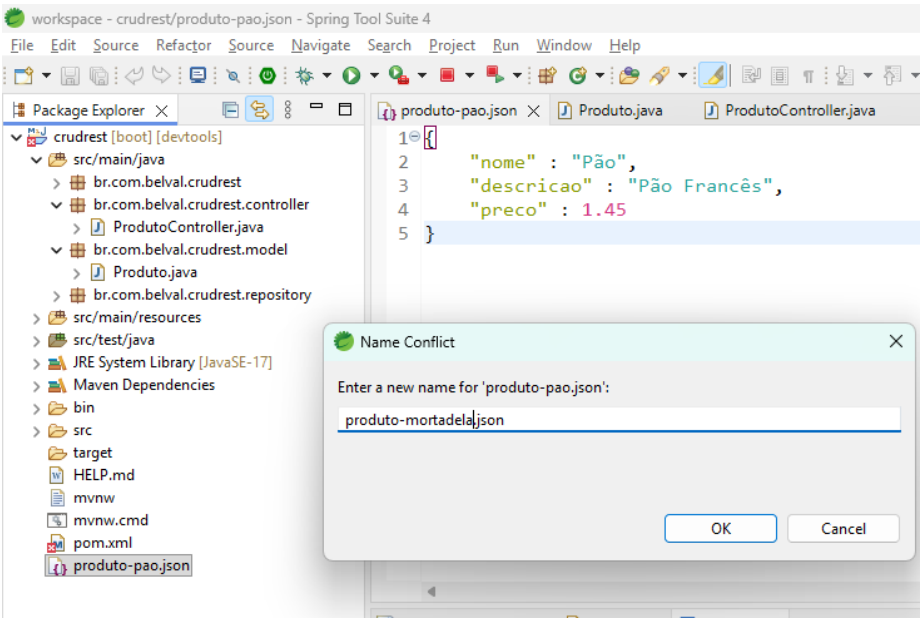
```
curl -X POST http://localhost:8080/produtos -H "Content-Type:
application/json; Charset=utf-8" -d @produto-pao.json
```

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ curl -X POST http://localhost:8080/produtos -H "Content-Type: application/json; Charset=utf-8" -d @produto-pao.json
{"dataCriacao":"2026-10-19T00:53:53.2209662","descricao":"Pão Francês","id":1,"nome":"Pão","preco":1.45}
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```

Se fizermos um select na tabela produto veremos o seguinte:

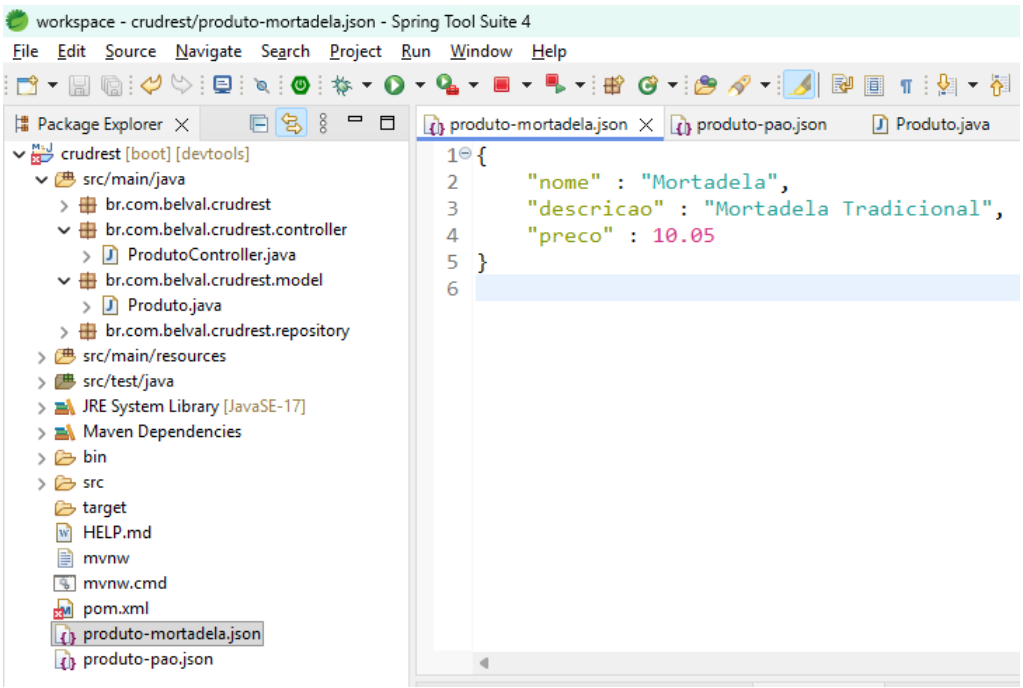


8.9 – Copie e cole o arquivo produto-pao.json e atribua o nome produto-mortadela.json para o arquivo copiado.



8.10 – Defina o conteúdo do arquivo produto-mortadela.json para o seguinte:

```
{
	"nome" : "Mortadela",
	"descricao" : "Mortadela Tradicional",
	"preco" : 10.05
}
```

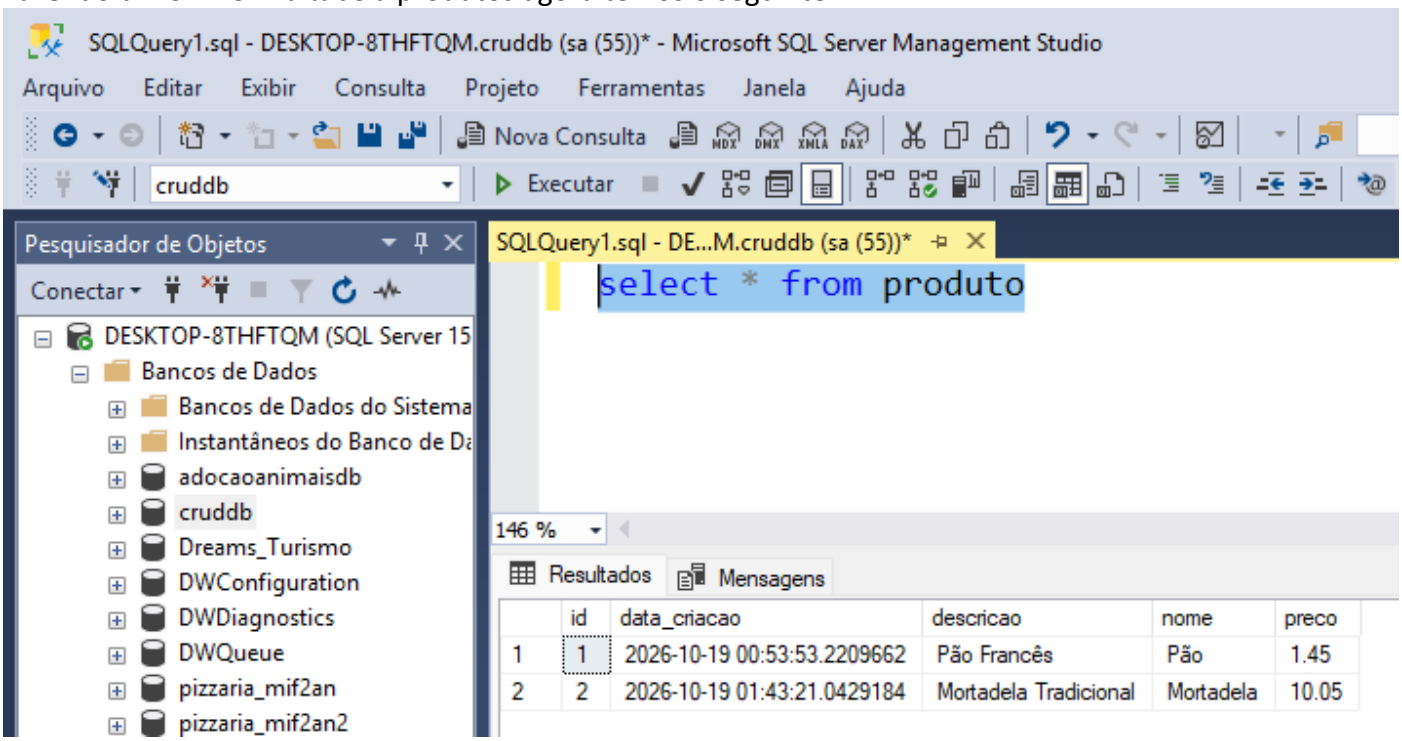


8.11 – Digite o seguinte comando no Git Bash e dê enter:

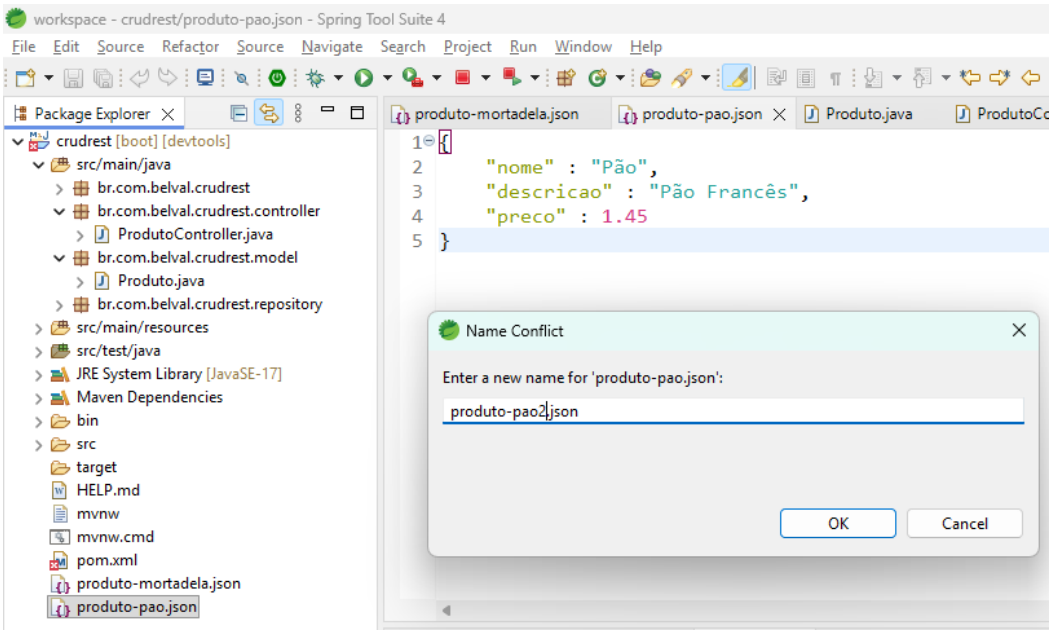
```
curl -X POST http://localhost:8080/produtos -H "Content-Type:
application/json; Charset=utf-8" -d @produto-mortadela.json
```

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ curl -X POST http://localhost:8080/produtos -H "Content-Type: application/json; Charset=utf-8" -d @produto-mortadela.jso
n
{"dataCriacao":"2026-10-19T01:43:21.0429184","descricao":"Mortadela Tradicional","id":2,"nome":"Mortadela","preco":10.05}
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```

Fazendo um SELECT na tabela produtos agora temos o seguinte:

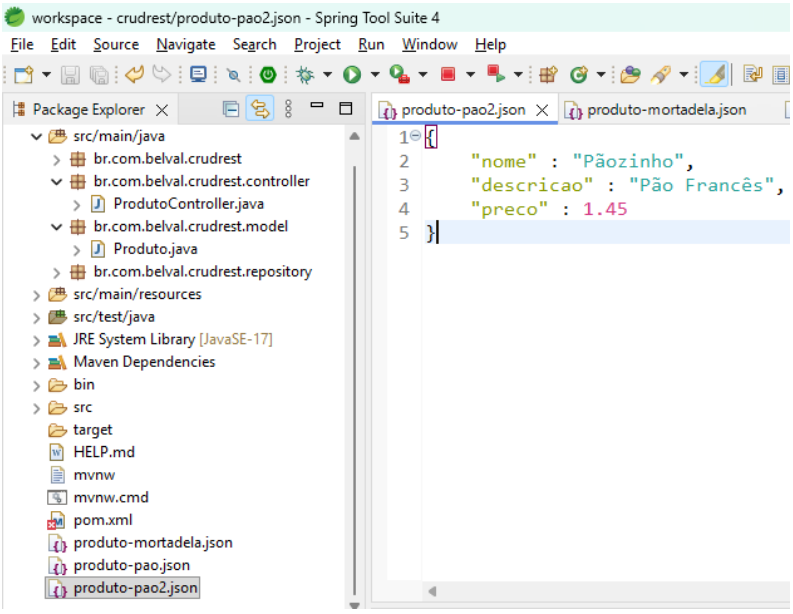


8.12 – Copie e cole o arquivo produto-pao.json e atribua o nome produto-pao2.json para o arquivo copiado.



8.13 – Defina o conteúdo do arquivo produto- pao2.json para o seguinte, alterando apenas o valor do campo nome:

```
{
	"nome" : "Pãozinho",
	"descricao" : "Pão Francês",
	"preco" : 1.45
}
```

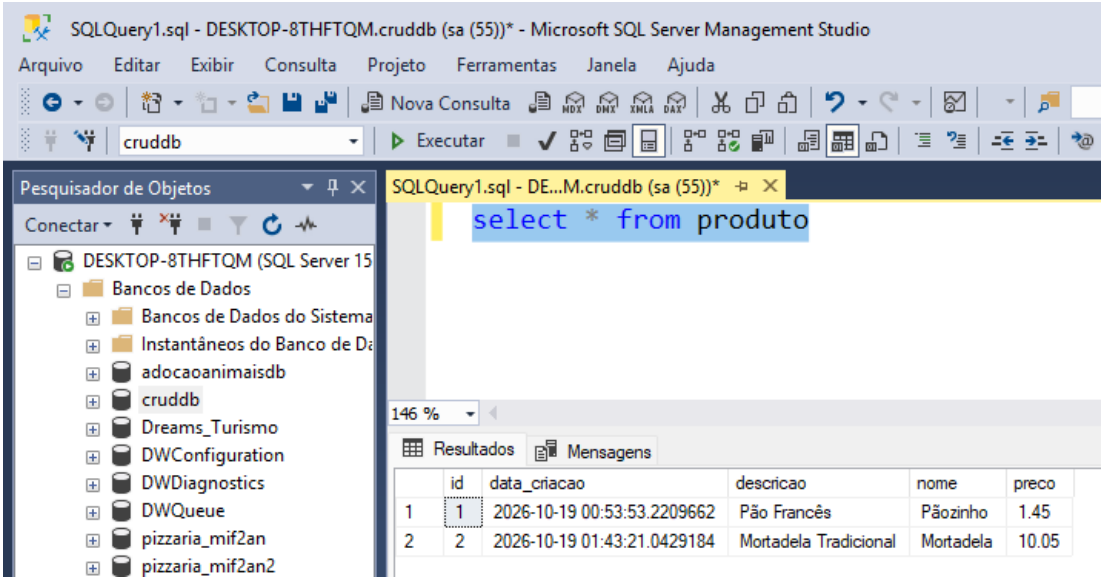


8.14 – Digite o seguinte comando no Git Bash e dê enter:

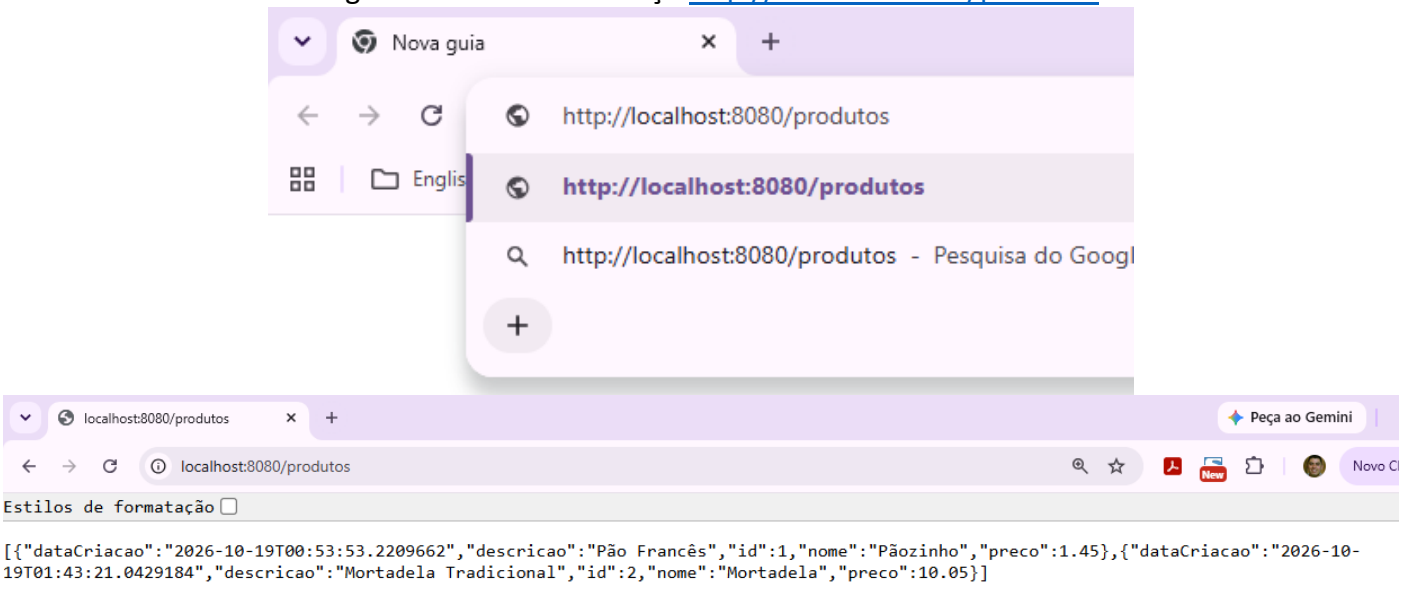
```
curl -X PUT http://localhost:8080/produtos/1 -H "Content-Type:
application/json; Charset=utf-8" -d @produto-pao2.json
```

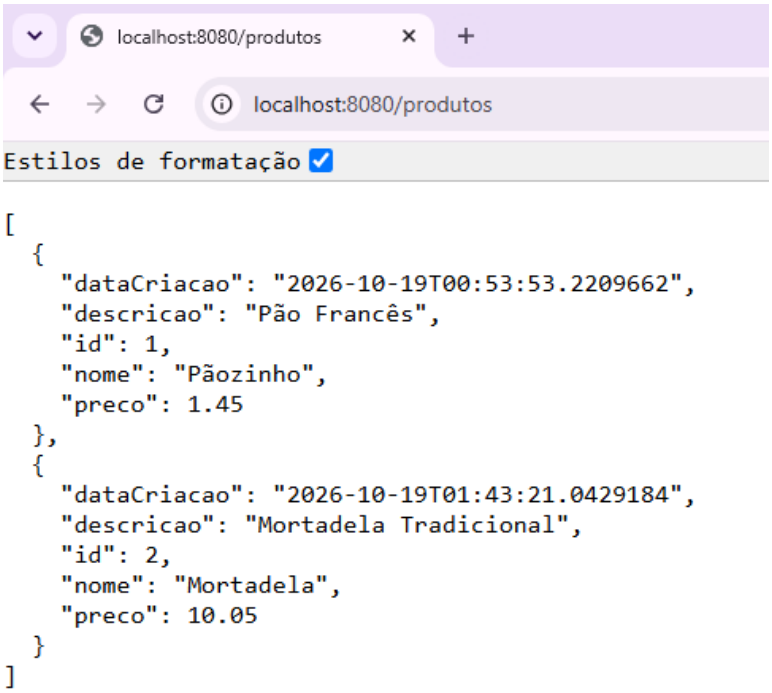
```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ curl -X PUT http://localhost:8080/produtos/1 -H "Content-Type: application/json; Charset=utf-8" -d @produto-pao2.json
Produto atualizado com sucesso!
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```

Fazendo um SELECT na tabela produtos agora temos o seguinte:

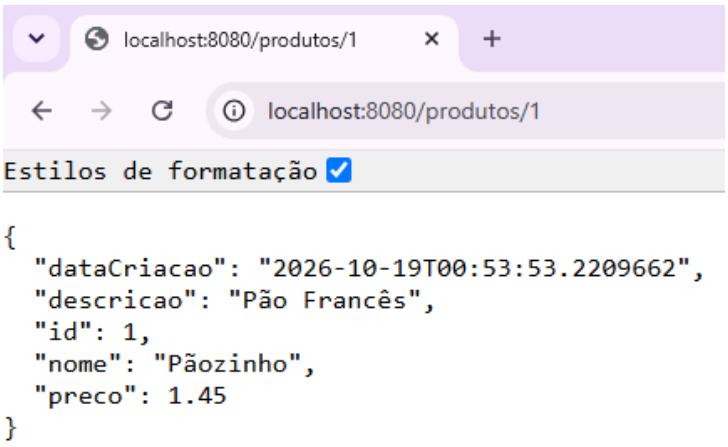


8.15 – Abra o browser e digite na barra de endereço http://localhost:8080/produtos:



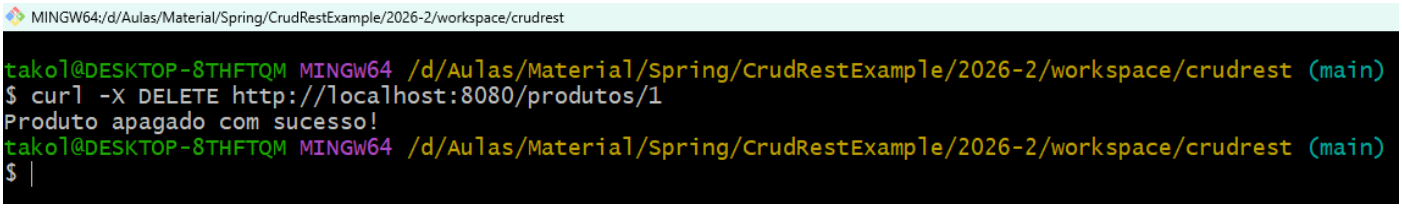


8.16 – Ainda no browser e digite na barra de endereço http://localhost:8080/produtos/1:

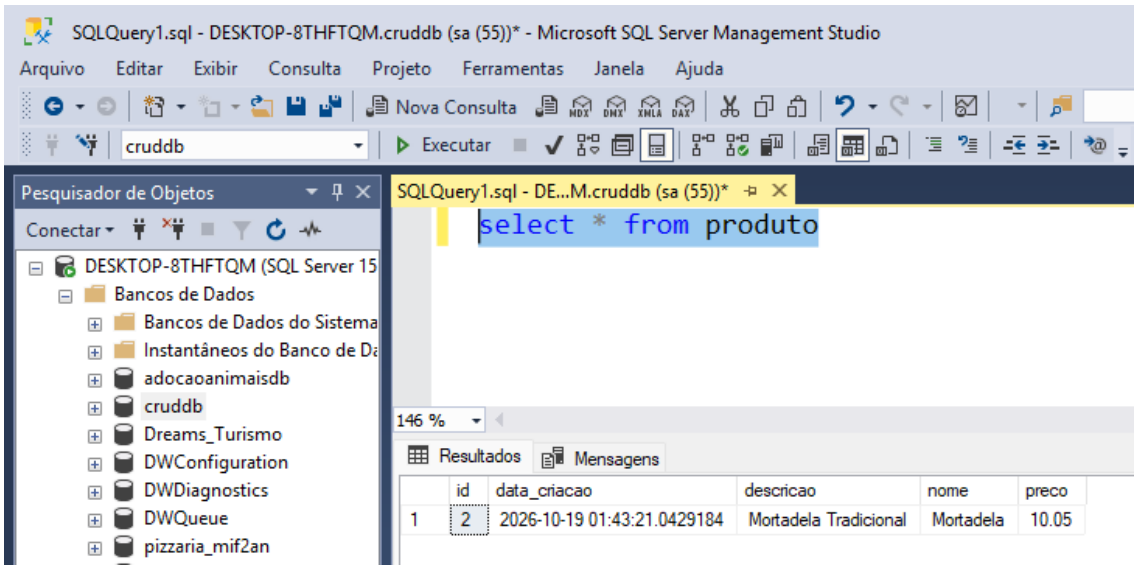


8.17 – Digite o seguinte comando no Git Bash e dê enter:

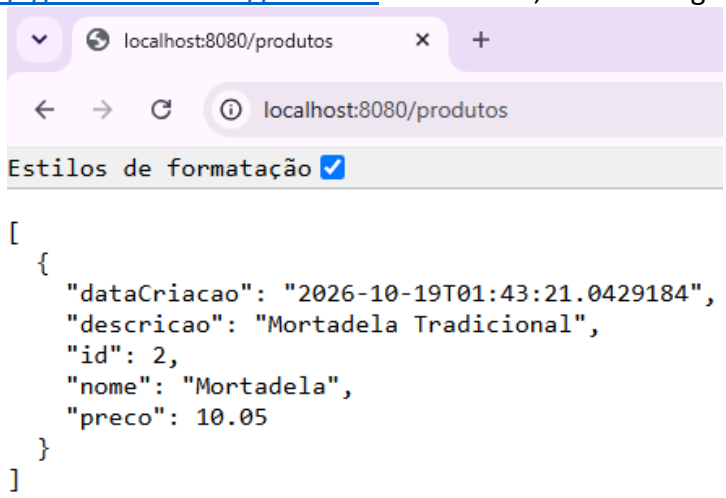
```
curl -X DELETE http://localhost:8080/produtos/1
```



Fazendo um SELECT na tabela produtos agora temos o seguinte:

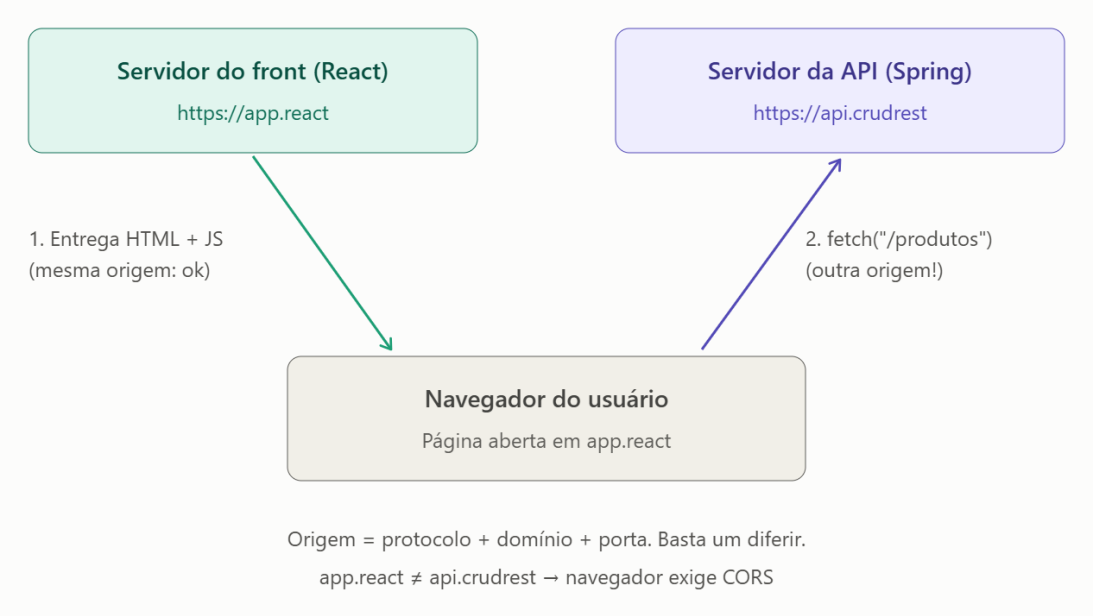


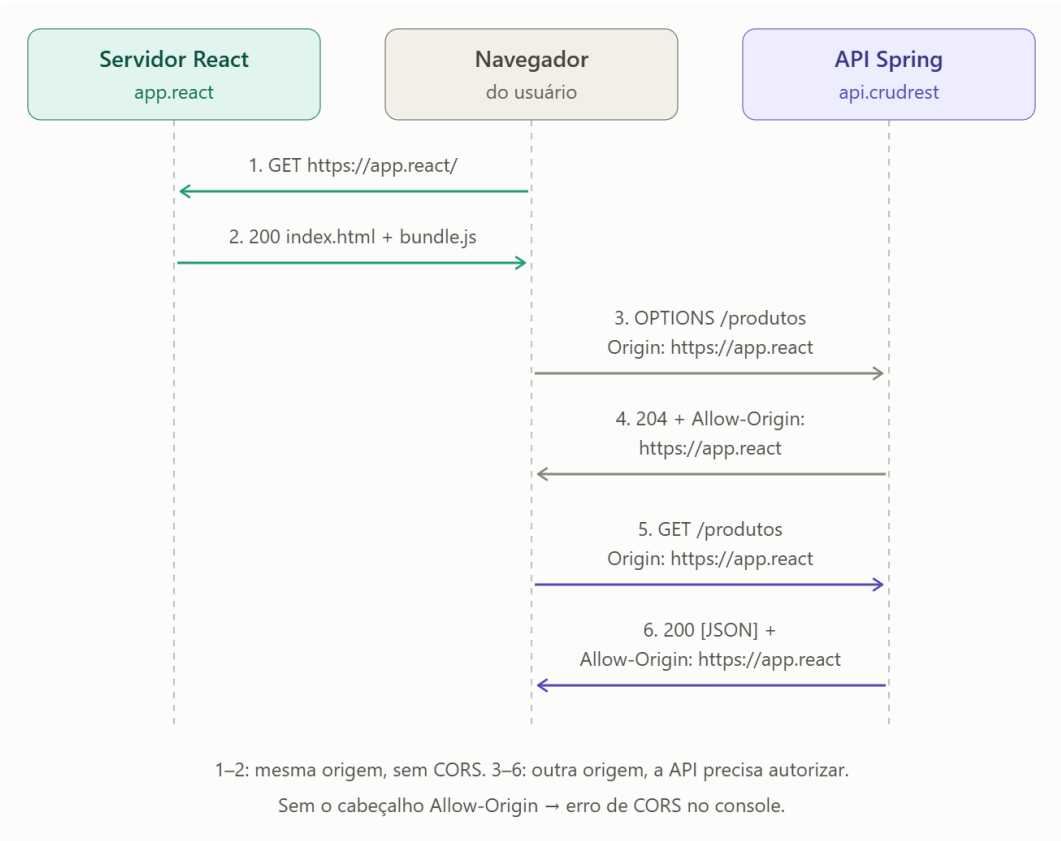
Acessando o endereço http://localhost:8080/produtos no browser, temos o seguinte:



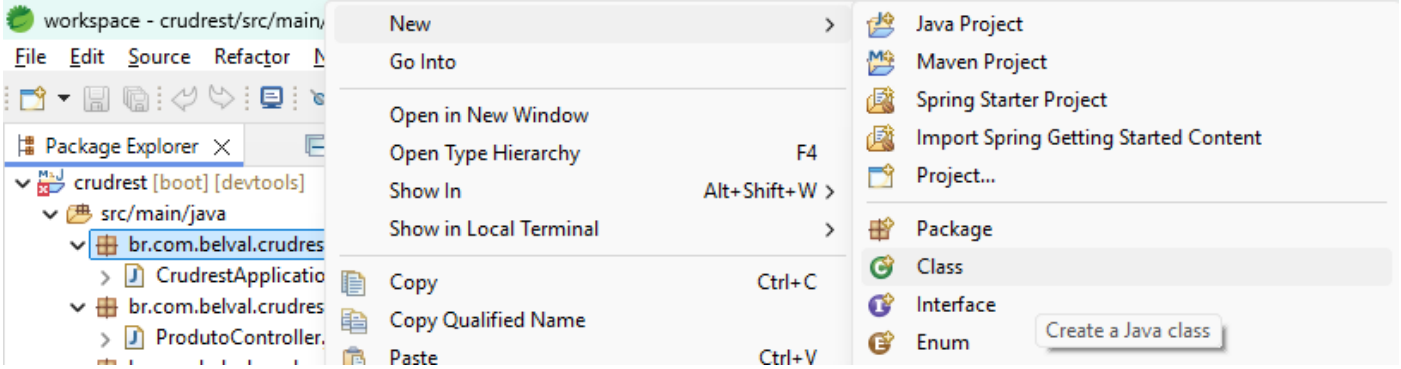
9 — Permitindo que uma aplicação React acesse nossa API

Uma aplicação React, normalmente, vai ficar hospedada em uma URL diferente de nossa API. Os browser's, por uma questão se segurança, tem por padrão bloquear acessos a URL's diferentes da URL de onde o browser carregou a aplicação cliente.

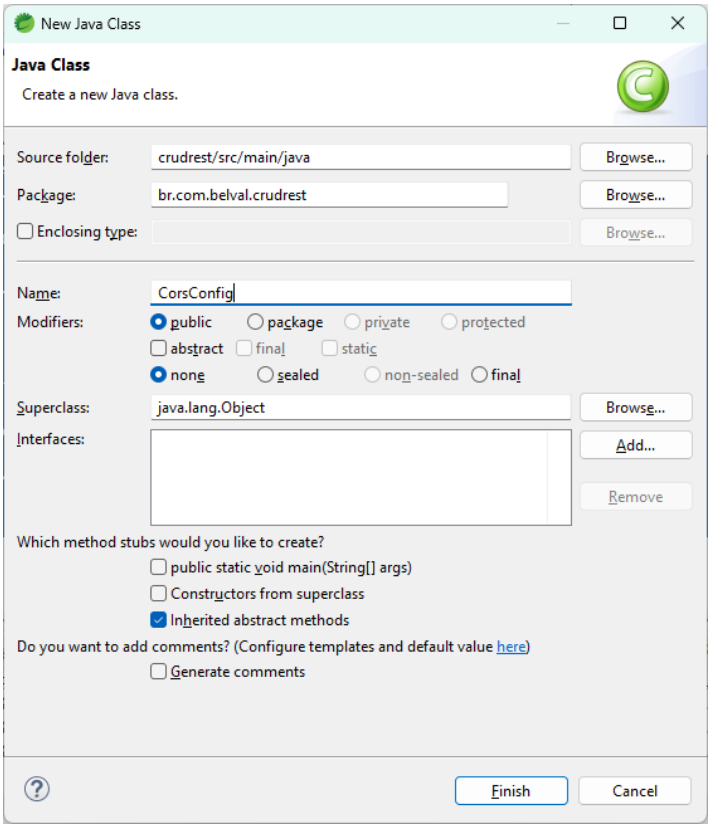


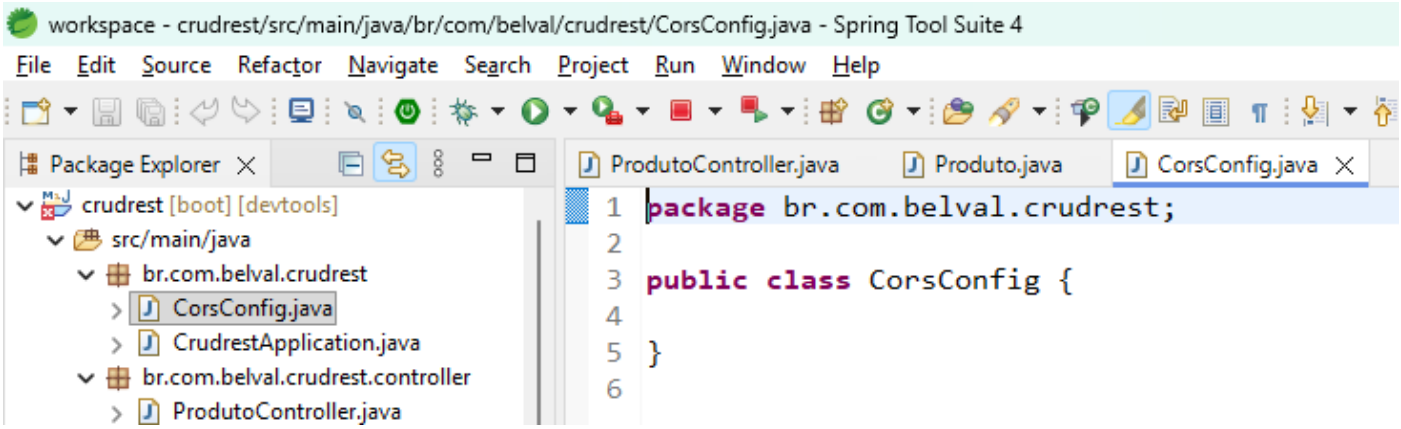


9.1 – Clique com o botão direito da aplicação no pacote br.com.belval.crudrest >> **New** >> **Class**



9.2 – Preencha o campo “Name” com “CorsConfig” e clique em “Finish”.





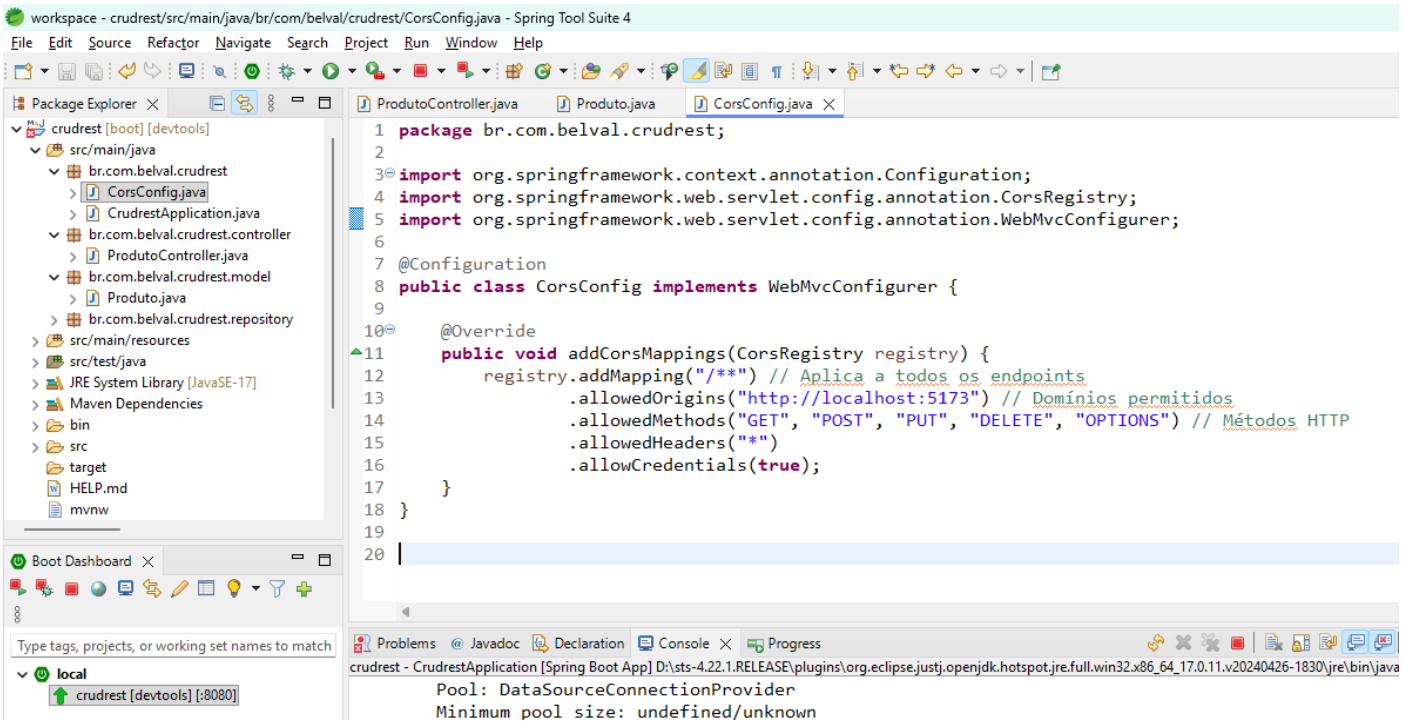
9.3 – Altere o código da classe CorsConfig para o seguinte:

```
package br.com.belval.crudrest;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class CorsConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**") // Aplica a todos os endpoints
                .allowedOrigins("http://localhost:5173") // Domínios permitidos
                .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS") // Métodos HTTP
                .allowedHeaders("*")
                .allowCredentials(true);
    }
}
```



10 — Commitando as alterações no Git

Após implementar todos os métodos, salve os arquivos e execute os comandos abaixo no Git Bash:

```
git status
git add .
git status
git commit -m "Add ProdutoController e faz as configurações de CORS para a API ser acessada pelo
React"
git push origin main
```

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        produto-mortadela.json
        produto-pao.json
        produto-pao2.json
        src/main/java/br/com/belval/crudrest/CorsConfig.java
        src/main/java/br/com/belval/crudrest/controller/

nothing added to commit but untracked files present (use "git add" to track)

takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git add .

takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   produto-mortadela.json
        new file:   produto-pao.json
        new file:   produto-pao2.json
        new file:   src/main/java/br/com/belval/crudrest/CorsConfig.java
        new file:   src/main/java/br/com/belval/crudrest/controller/ProdutoController.java


takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git commit -m "Add ProdutoController e faz as configurações de CORS para a API ser acessada pelo React"
[main 5fdbfc6] Add ProdutoController e faz as configurações de CORS para a API ser acessada pelo React
 5 files changed, 146 insertions(+)
 create mode 100644 produto-mortadela.json
 create mode 100644 produto-pao.json
 create mode 100644 produto-pao2.json
 create mode 100644 src/main/java/br/com/belval/crudrest/CorsConfig.java
 create mode 100644 src/main/java/br/com/belval/crudrest/controller/ProdutoController.java

takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git push origin main
Enumerating objects: 23, done.
Counting objects: 100% (23/23), done.
Delta compression using up to 8 threads
Compressing objects: 100% (11/11), done.
Writing objects: 100% (15/15), 2.53 KiB | 862.00 KiB/s, done.
Total 15 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/ProfNpc/crudrest.git
   400cb2a..5fdbfc6  main -> main

takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```

Resumo do que foi feito nesta parte

- Entendemos o papel do `@RestController` e da anotação `@RequestMapping` na definição de endpoints HTTP.
- Injetamos o `ProdutoRepository` no controller via `@Autowired`.
- Implementamos `criarProduto` (`POST /produtos`) com `@PostMapping` e `@RequestBody`.
- Implementamos `obterProduto` (`GET /produtos/{id}`) com `@GetMapping` e `@PathVariable`.
- Implementamos `listarProdutos` (`GET /produtos`) retornando a lista completa.
- Implementamos `atualizarProduto` (`PUT /produtos/{id}`) atualizando campo a campo o produto existente.
- Implementamos `apagarProduto` (`DELETE /produtos/{id}`) com remoção física do banco.
- Entendemos que o que provoca os erros de CORS (*Cross-Origin Resource Sharing* ou Compartilhamento de Recursos entre Origens) é um mecanismo de segurança implementado pelos navegadores. Ele controla se uma aplicação web em um domínio pode fazer requisições para uma API ou recurso em outro domínio.